

Maestría
en
OpenAI API

Guía Completa para Todos los Niveles

Junio/2024

Introducción

"La inteligencia artificial no reemplazará a los humanos, pero aquellos que la dominen reemplazarán a aquellos que no." - Unknown

Bienvenido a "Maestría en OpenAI API: Guía Completa para Todos los Niveles". Este libro te llevará a través de un emocionante viaje para dominar las capacidades de la API de OpenAI, una herramienta poderosa que está transformando la manera en que interactuamos con la tecnología. Desde los conceptos básicos hasta las aplicaciones más avanzadas, este libro está diseñado para proporcionarte los conocimientos y habilidades necesarios para aprovechar al máximo el potencial de la inteligencia artificial.

Resumen

Este libro es una guía exhaustiva que te llevará desde los conceptos básicos hasta las aplicaciones avanzadas de la API de OpenAI. Aprenderás a generar texto, realizar análisis de sentimientos, crear chatbots inteligentes, y mucho más. Cada capítulo está diseñado para ser claro y directo, siguiendo la teoría de Iceberg, asegurando que la información sea accesible tanto para principiantes como para expertos. Con ejemplos prácticos y explicaciones detalladas, este libro te proporcionará las herramientas necesarias para aprovechar al máximo el potencial de OpenAI API.

¿Para Quién es Este Libro?

Este libro está dirigido a:

- **Desarrolladores y Programadores:** Que buscan integrar capacidades avanzadas de inteligencia artificial en sus aplicaciones.
- **Estudiantes de IA y Aprendizaje Automático:** Que desean entender y utilizar OpenAI API en sus proyectos académicos.
- **Emprendedores y Empresarios:** Que quieren explorar cómo la inteligencia artificial puede mejorar sus productos y servicios.
- **Entusiastas de la Tecnología:** Que tienen curiosidad por las aplicaciones prácticas de la inteligencia artificial en el mundo real.

Prerrequisitos

Antes de comenzar con este libro, se recomienda tener:

- **Conocimientos Básicos de Programación:** Familiaridad con algún lenguaje de programación, preferiblemente Python.
- **Entorno de Desarrollo Configurado:** Tener instalado Python, un editor de texto o IDE (como Visual Studio Code), y Android Studio si se planea trabajar con aplicaciones móviles.
- **Cuenta de OpenAI:** Registrarse en OpenAI y obtener una clave API para realizar las solicitudes y ejemplos prácticos.

Capítulos del Libro

Capítulo 1: Primeros Pasos con OpenAI API

- [¿Qué es OpenAI API?](#)
- [Configuración Inicial](#)

Capítulo 2: Conceptos Básicos

- [Fundamentos de OpenAI API](#)
- [Ejemplos Prácticos](#)

Capítulo 3: Explorando Capacidades

- [Generación de Texto](#)
- [Manejo de Conversaciones](#)
- [Controlando el Tono y Estilo](#)

Capítulo 4: Aplicaciones Avanzadas

- [Análisis de Sentimientos](#)
- [Generación de Código](#)
- [Resumen Automático de Textos](#)

Capítulo 5: Optimización y Mejores Prácticas

- [Optimización de Modelos](#)
- [Mejores Prácticas de Desarrollo](#)

Capítulo 6: Casos de Estudio

- [Ejemplos Reales de Uso](#)
- [Lecciones Aprendidas](#)

Capítulo 7: Técnicas Avanzadas

- [Fine-Tuning de Modelos](#)
- [Cargar Archivos y Obtener Resultados](#)
- [Calling Functions - Llamando Funciones](#)
- [Cargar Imágenes y Describirlas \(Reconocimiento Visual\)](#)
- [Convertir Texto en Audio y Viceversa](#)
- [Crear Videos a Través de Texto con el Modelo Sora](#)

Capítulo 8: Integración en Proyectos Reales

- [Integración con Aplicaciones Web](#)
- [Integración con Aplicaciones Móviles](#)

- [Crear una Aplicación de Chatbot con Android Studio y Jetpack Compose](#)
- [Crear una Aplicación para Smart Watch con Wear OS y Jetpack Compose](#)
- [Crear una Aplicación para Automóviles con Jetpack Compose](#)

Capítulo 9: Optimización y Mejores Prácticas Avanzadas

- [Estrategias Avanzadas de Optimización](#)
- [Seguridad y Escalabilidad](#)

Capítulo 10: Recursos Adicionales y Comunidad

- [Recursos para Continuar Aprendiendo](#)
- [Participación en la Comunidad](#)

Capítulo 11: Conclusión

- [Resumen de Claves Aprendidas](#)
- [Mirando Hacia el Futuro](#)

Preguntas Frecuentes (FAQ)

Plan de Estudio Sugerido

Recursos Adicionales

Descripción del Autor

Apendice

Capítulo 1:

Primeros Pasos con OpenAI API

¿Qué es OpenAI API?

OpenAI API es una interfaz de programación de aplicaciones que permite a los desarrolladores integrar modelos de inteligencia artificial avanzados en sus aplicaciones. Desarrollada por OpenAI, esta API facilita tareas como la generación de texto, la traducción, el análisis de sentimientos, entre muchas otras.

Características Principales:

- **Acceso a Modelos Avanzados:** Utiliza modelos de última generación como GPT-3.
- **Versatilidad:** Aplica la IA en diversas áreas como chatbots, generación de contenido y más.
- **Fácil Integración:** Integración sencilla con varias plataformas y lenguajes de programación.

Casos de Uso Populares:

- **Chatbots y Asistentes Virtuales:** Mejora la interacción con los usuarios.
- **Generación Automática de Contenido:** Crea artículos, resúmenes y más de manera eficiente.
- **Análisis de Datos:** Extrae información valiosa de grandes volúmenes de texto.

Configuración Inicial

Cómo Obtener Acceso a OpenAI API:

1. **Registro:** Visita el sitio web de OpenAI y regístrate para obtener una cuenta.
2. **Obtención de la Clave API:** Después de registrarte, recibirás una clave API. Esta clave es necesaria para realizar solicitudes a la API.

Instalación y Configuración del Entorno:

Para comenzar a usar OpenAI API, necesitas instalar las bibliotecas necesarias en tu entorno de desarrollo. A continuación, te mostramos cómo hacerlo en Python.

Instalación de la Biblioteca OpenAI:

```
pip install openai
```

Configuración de la Clave API:

```
import openai  
  
# Reemplaza 'tu-api-key' con tu clave API de OpenAI  
openai.api_key = 'tu-api-key'
```

Primer Ejemplo de Solicitud:

Ahora que tienes todo configurado, hagamos una solicitud básica para ver cómo funciona.

```
import openai

openai.api_key = 'tu-api-key'

response = openai.chat.completions.create(

    model="gpt-3.5-turbo",

    messages=[

        {"role": "user", "content": "Hola, como  
estas?"}

    ]

)

print(response.choices[0].message.content.strip())
```

Salida:

¡Hola! Estoy bien, ¿cómo estás tú? ¿En qué puedo ayudarte hoy?

Este código hace una solicitud a la API de OpenAI y genera una respuesta basada en el prompt proporcionado. Es así de sencillo comenzar a interactuar con la inteligencia artificial de OpenAI.

Notas Adicionales:

A lo largo del libro, cada capítulo contendrá ejemplos prácticos como este, junto con explicaciones claras y concisas. Utilizaremos un enfoque progresivo, comenzando con conceptos básicos y avanzando hacia aplicaciones más complejas y optimizaciones.

Capítulo 2: Conceptos Básicos

Fundamentos de OpenAI API

OpenAI API ofrece varios modelos diseñados para diferentes tareas. A continuación, describimos los modelos principales y sus usos.

Modelos Principales:

1. **GPT-3 (Generative Pre-trained Transformer 3):**
 - **Uso:** Generación de texto coherente y creativo.
 - **Características:** Capacidad de completar frases, responder preguntas, escribir artículos y más.
2. **Codex:**
 - **Uso:** Generación de código y ayuda en tareas de programación.
 - **Características:** Puede escribir, completar y depurar código en varios lenguajes de programación.
3. **DALL-E:**
 - **Uso:** Generación de imágenes a partir de descripciones textuales.
 - **Características:** Capacidad de crear imágenes nuevas y originales basadas en prompts textuales.
4. **Whisper:**
 - **Uso:** Transcripción de audio a texto.
 - **Características:** Alta precisión en la conversión de voz a texto en múltiples idiomas.

Ejemplos Prácticos

Generación de Texto con GPT-3

Para generar texto con GPT-3, usaremos el siguiente ejemplo sencillo:

```
import openai

openai.api_key = 'tu-api-key'

response = openai.chat.completions.create(

    model="gpt-3.5-turbo",

    messages=[

        {"role": "user", "content": "Escribe la
        historia de un gato"

    },

    max_tokens = 100

)

print(response.choices[0].message.content.strip())
```

Este código envía una solicitud a GPT-3 para generar una historia sobre un gato aventurero. La respuesta incluye un texto coherente y creativo.

Salida:

Había una vez un gato llamado Oliver, que era conocido en toda la vecindad por ser el más aventurero de todos. Desde pequeño, le gustaba explorar cada rincón de la ciudad y siempre se metía en problemas por su curiosidad.

Un día, Oliver decidió que quería explorar más allá de los límites de la ciudad y emprendió un viaje hacia tierras desconocidas. Durante su ...

Generación de Código con Codex

Codex es ideal para tareas de programación. Aquí tienes un ejemplo:

```
import openai

openai.api_key = 'tu-api-key'

import openai

openai.api_key = 'tu-api-key'

response = openai.chat.completions.create(

    model="gpt-3.5-turbo",

    messages=[

        {"role": "user", "content": "Escribe una
función en Python que ordene una lista de números."

    },

    max_tokens = 100)

print(response.choices[0].message.content.strip())
```

Salida:

```
def ordenar_lista(lista):
    lista_ordenada = sorted(lista)
    return lista_ordenada

# Ejemplo de uso
numeros = [5, 2, 8, 1, 7]
numeros_ordenados = ordenar_lista(numeros)
print(numeros_ordenados) # Output: [1, 2, 5, 7, 8]
```

Este ejemplo muestra cómo Codex puede generar una función en Python para ordenar una lista de números.

Generación de Imágenes con DALL-E

DALL-E genera imágenes a partir de descripciones textuales. Aquí tienes un ejemplo:

```
import openai

openai.api_key = 'tu-api-key'

response = openai.images.generate(
    model="dall-e-3",
    prompt="a white siamese cat",
    size="1024x1024",
    quality="standard",
    n=1,
)
image_url = response.data[0].url
print(image_url)
```

Este código envía una solicitud a DALL-E para crear una imagen de un gato aventurero explorando una jungla. La respuesta incluye una URL con la imagen generada.

Salida



Transcripción de Audio con Whisper

Whisper convierte texto a audio y vice-versa con alta precisión. Aquí tienes ejemplos:

Convertir Texto a Audio

```
import openai

openai.api_key = 'tu-api-key'

response = client.audio.speech.create(
    model="tts-1",
    voice="alloy",
    input="Hello world! This is a streaming test.",
)

response.write_to_file("output.mp3")
```

Este código genera el archivo “**output.mp3**”

Nota:

Para más detalles explore:

<https://platform.openai.com/docs/guides/speech-to-text/quickstart>

Convertir Audio to Texto

```
import openai

openai.api_key = 'tu-api-key'

audio_file = open("output.mp3", "rb")

transcription = openai.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file
)

print(transcription.text)
```

Este ejemplo muestra cómo Whisper transcribe un archivo de audio a texto.

Hello, world. This is a streaming test.

Autenticación y Seguridad

Para interactuar con la API de OpenAI, es crucial autenticarse correctamente y manejar la seguridad de la clave API.

Autenticación Básica:

Tu clave API es necesaria para todas las solicitudes a OpenAI. Asegúrate de mantenerla segura y no compartirla públicamente.

```
import openai
```

```
openai.api_key = 'tu-api-key'
```

Seguridad de la Clave API:

- No compartas tu clave públicamente.
- Utiliza variables de entorno para almacenar la clave de manera segura.
- Revoca y reemplaza la clave si crees que ha sido comprometida.

Nota de Seguridad: Por razones de objetividad, los ejemplos mostraron las API keys de forma insegura. Se recomienda almacenarlas en variables de entorno.

Pasos para Configurar la API de OpenAI de Forma Permanente:

1. Obtén tu API Key:

- Visita la página de OpenAI y crea una cuenta.
- Genera una API key en tu panel de control.

2. Instala la Librería de OpenAI:

Usa pip para instalar la librería oficial:

bash

```
pip install openai
```

3. Configura la Variable de Entorno:

- **En sistemas Unix (Linux/macOS):**
 - Abre el archivo de configuración de tu shell (por ejemplo, `~/.bashrc` o `~/.zshrc`).

Añade la siguiente línea:

```
export OPENAI_API_KEY='tu-api-key-aquí'
```

- Guarda el archivo y ejecuta `source ~/.bashrc` (o el archivo que hayas editado).
- **En Windows:**
 - Abre las configuraciones de Variables de Entorno del sistema.
 - Crea una nueva variable de usuario llamada `OPENAI_API_KEY` con tu API key como valor.
 - Reinicia tu sesión o la computadora para que los cambios surtan efecto.

2. Verifica la Configuración:

- Abre una nueva terminal o prompt de comandos.

Ejecuta Python y prueba la configuración:

```
python
import os
import openai

openai.api_key = os.getenv("OPENAI_API_KEY")
# Intenta hacer una llamada a la API para verificar
```

3. Implementa Buenas Prácticas de Seguridad:

- Nunca compartas tu API key públicamente.
- Usa un gestor de secretos para mayor seguridad.
- Rota tus API keys periódicamente.

4. Manejo de Errores y Límites de Uso:

- Implementa manejo de errores en tu código para lidiar con problemas de conexión o límites de uso.
- Monitorea tu uso de la API para evitar costos inesperados.

Resumen

Al seguir estos pasos, habrás configurado de manera segura y permanente tu entorno para usar la API de OpenAI. La parte visible (obtener la key e instalar la librería) es solo la punta del iceberg, mientras que la configuración del entorno, las prácticas de seguridad y el manejo de errores forman la base sólida y menos visible pero crucial para un uso efectivo y seguro de la API.

Capítulo 3: Explorando Capacidades

Generación de Texto

La generación de texto es una de las capacidades más potentes de OpenAI API. A continuación, exploramos cómo crear texto de manera coherente y controlada.

Ejemplo de Generación de Texto:

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a helpful
assistant."},
        {"role": "user", "content": "Who won the world series in
2020?"},
        {"role": "assistant", "content": "The Los Angeles Dodgers
won the World Series in 2020."},
        {"role": "user", "content": "Where was it played?"}
    ]
)

print(response.choices[0].message.content.strip())
```

Salida:

```
The World Series in 2020 was played at a neutral site due to the
COVID-19 pandemic. The games were held at Globe Life Field in
Arlington, Texas.
```

Explicacion del código detalladamente:

```
from openai import OpenAI:
```

Esta línea importa la clase OpenAI del módulo openai. Esta es la nueva forma de importar la biblioteca de OpenAI para usar su API.

```
client = OpenAI():
```

Crea una instancia del cliente OpenAI. Por defecto, buscará la clave API en la variable de entorno OPENAI_API_KEY.

```
response = client.chat.completions.create(...):
```

Esta línea crea una nueva conversación de chat utilizando el modelo de lenguaje de OpenAI.

```
model="gpt-3.5-turbo":
```

Especifica qué modelo de lenguaje se utilizará. En este caso, es GPT-3.5 Turbo, que es eficiente y capaz de mantener conversaciones.

```
messages=[...]:
```

Esta es una lista de mensajes que forman la conversación. Cada mensaje es un diccionario con dos claves: "role" y "content".

Los mensajes en la lista:

```
{"role": "system", "content": "You are a helpful assistant."}:
```

Este es un mensaje de sistema que establece el contexto para el asistente.

```
{"role": "user", "content": "Who won the world series in 2020?"}:
```

Este es el primer mensaje del usuario, haciendo una pregunta.

```
{"role": "assistant", "content": "The Los Angeles Dodgers won the World Series in 2020."}
```

Esta es la respuesta del asistente a la primera pregunta.

```
{"role": "user", "content": "Where was it played?"}
```

Este es el segundo mensaje del usuario, haciendo una pregunta de seguimiento.

```
print(response.choices.message.content.strip()):
```

response.choices: Contiene las respuestas generadas por el modelo.

``: Selecciona la primera (y generalmente única) respuesta.

.message.content: Obtiene el contenido del mensaje de la respuesta.

.strip(): Elimina cualquier espacio en blanco al principio o al final de la respuesta.

print(): Imprime la respuesta en la consola.

Este código simula una conversación con el modelo GPT-3.5 Turbo. Primero, se le da un contexto al modelo ("You are a helpful assistant."). Luego, se le hace una pregunta sobre la Serie Mundial de 2020, se incluye una respuesta a esa pregunta, y finalmente se hace una pregunta de seguimiento sobre dónde se jugó. El modelo utilizará todo este contexto para generar una respuesta apropiada a la última pregunta.

En una aplicación real podríamos hacerlo de la siguiente manera:

```

from openai import OpenAI
client = OpenAI()

messages=[
    {"role": "system", "content": "You are a helpful
assistant."},
    {"role": "user", "content": "My name is Denis. How are you,
today?"},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages
    )

new_question = "Escribe una poesia acerca del amor."
new_raw = {
    "role":"user",
    "content": new_question
}
messages.append(new_raw)

response = getResponse(messages=messages)

print(response.choices[0].message.content)

```

La Salida seria:

En el suave susurro de la brisa nocturna,
nace un amor eterno, profundo como la Luna.
Un vínculo sagrado, que trasciende el tiempo,
un sentimiento puro, en cada momento.

En el abrazo cálido de dos almas gemelas,
se entrelazan los destinos en las estrellas.
El corazón late al compás de la pasión,
creando un lazo indeleble, sin comparación.

El amor es un poema que se escribe en el aire,
una melodía sublime que nos invita a soñar.
Es el lazo invisible que une corazones,
trayendo luz a nuestras vidas en todas direcciones.

Que florezca el amor en cada latido,
que brille en los ojos y en cada suspiro.
Que sea el faro que guíe nuestro camino,
iluminando el sendero con su divino destino.

Explicacion del código detalladamente:

```
from openai import OpenAI:
```

Importa la clase OpenAI del módulo openai, necesaria para interactuar con la API de OpenAI.

```
client = OpenAI():
```

Crea una instancia del cliente OpenAI. Por defecto, buscará la clave API en la variable de entorno OPENAI_API_KEY.

```
messages=[...]:
```

Define una lista de mensajes que inicializa la conversación. Contiene dos mensajes iniciales:

Un mensaje de sistema que establece el rol del asistente.

Un mensaje del usuario que introduce al usuario (Denis) y saluda al asistente.

```
def getResponse(messages)::
```

Define una función llamada getResponse que toma como argumento la lista de mensajes.

Esta función crea una nueva completación de chat usando el modelo "gpt-3.5-turbo" y los mensajes proporcionados.

Retorna la respuesta del modelo.

```
new_question = "Escribe una poesia acerca del amor.":
```

Define una nueva pregunta o solicitud para el asistente.

```
new_raw = {...}:
```

Crea un nuevo diccionario que representa un mensaje del usuario con la nueva pregunta.

```
messages.append(new_raw):
```

Añade el nuevo mensaje del usuario a la lista de mensajes existente.

```
response = getResponse(messages=messages):
```

Llama a la función `getResponse` con la lista actualizada de mensajes.

Almacena la respuesta del modelo en la variable `response`.

```
print(response.choices.message.content):
```

Imprime el contenido del mensaje de la primera (y generalmente única) opción de respuesta.

Este código hace lo siguiente:

Establece una conversación inicial con el asistente.

Define una función para obtener respuestas del modelo.

Añade una nueva pregunta a la conversación (solicitando una poesía sobre el amor).

Obtiene una respuesta del modelo para esta nueva solicitud.

Imprime la respuesta (que debería ser una poesía sobre el amor).

La estructura permite mantener un contexto de conversación, donde el modelo puede referirse a información previa (como el nombre del usuario) al generar nuevas respuestas.

Manejo de Conversaciones

Crear un chatbot inteligente es una de las aplicaciones más populares de OpenAI API. Aquí se muestra cómo mantener el contexto en una conversación.

Ejemplo de Chatbot Básico:

```
from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "My name is Denis. How are you, today?"},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages
    )

def print_assistant_response(response):
    print("\nAssistant:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
    initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta (o 's' / 'x' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print("¡Hasta luego!")
        break
```

```
# Añadir la pregunta del usuario a los mensajes
messages.append({"role": "user", "content": user_input})

# Obtener y mostrar la respuesta del asistente
response = getResponse(messages)
print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content})
```

La salida:

Assistant: Hello Denis! I'm just a digital assistant, so I don't have feelings, but I'm here and ready to assist you. How can I help you today?

Escribe tu pregunta ('x' para salir): Escribe un cuento corto de un niño y un perro

Assistant: Había una vez un niño llamado Diego que vivía en un pequeño pueblo junto a su fiel compañero, un perro llamado Max. Diego y Max eran inseparables, pasaban todo el día juntos explorando el bosque que rodeaba su casa.

Un día, mientras caminaban por el bosque, Max comenzó a ladrar emocionado y corrió hacia un árbol. Diego lo siguió y descubrió a un pequeño cachorro de zorro atrapado en una rama alta. Sin dudar, Diego trepó ágilmente por el árbol y rescató al cachorro.

A partir de ese día, el cachorro de zorro se unió a la aventura de Diego y Max. Juntos exploraban el bosque, jugaban y vivían muchas aventuras emocionantes. La amistad entre el niño, el perro y el zorro se fortaleció con cada día que pasaba.

Y así, Diego, Max y el cachorro de zorro se convirtieron en los mejores amigos, demostrando que la verdadera amistad no conoce barreras ni diferencias, y que juntos podían enfrentar cualquier desafío que se les presentara.

Escribe tu pregunta ('x' para salir): muy bien, me gusta

Assistant: Me alegra que te haya gustado el cuento. Si necesitas algo más, no dudes en pedirlo. ¡Estoy aquí para ayudarte en lo que necesites!

Escribe tu pregunta ('x' para salir): No, por ahora estoy bien. Gracias.

Assistant: ¡Perfecto! No dudes en volver si necesitas ayuda en el futuro. ¡Que tengas un buen día, Denis!

Escribe tu pregunta ('x' para salir): x

¡Hasta luego!

Este ejemplo muestra cómo mantener el historial de la conversación para que el chatbot pueda responder de manera coherente.

Controlando el Tono y Estilo

Puedes ajustar el tono y estilo de las respuestas controlando los parámetros de la solicitud.

Ejemplo de Control del Tono:

```
from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "My name is Denis. How are you, today?"},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
        max_tokens=50,
        temperature=0.7, # Controla la creatividad
        top_p=0.9         # Controla la deversidad
    )

def print_assistant_response(response):
    print("\nAssistant:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

while True:
```

```

user_input = input("\nEscribe tu pregunta (o 's' / 'x' para salir): ")

if user_input.lower() in ['s', 'x']:
    print("¡Hasta luego!")
    break

# Añadir la pregunta del usuario a los mensajes
messages.append({"role": "user", "content": user_input})

# Obtener y mostrar la respuesta del asistente
response = getResponse(messages)
print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content})

```

Salida:

Assistant: Hello Denis! I'm just a computer program, so I don't have feelings, but I'm here and ready to help you. How can I assist you today?

Escribe tu pregunta ('x' para salir): Quiero que seas creativo y me digas algo gracioso

Assistant: ¡Claro que sí! Aquí va uno: ¿Por qué los pájaros no usan Facebook? Porque ya tienen Twitter. ¡Espero que te haya sacado una sonrisa! ¿Hay algo más en lo que

Escribe tu pregunta ('x' para salir): Que gracioso! Intentalo de nuevo. 🙄

Assistant: ¡Claro! Aquí va otro: ¿Por qué los esqueletos no pelean entre ellos? Porque no tienen agallas. ¡Espero que este también te haya hecho reír! ¿Puedo ayudarte

Escribe tu pregunta ('x' para salir): Uno mas y terminamos 🙄

Assistant: ¡Por supuesto! Aquí tienes otro chiste: ¿Por qué el libro de matemáticas estaba triste? Porque tenía demasiados problemas. ¡Espero que te haya gustado! ¿Hay algo más en lo que pueda

Escribe tu pregunta ('x' para salir): Ese si me gusto. Felicidades. Lo lograste. 🎉

Assistant: ¡Me alegra saber que te hizo reír! Si necesitas ayuda con algo más en el futuro, no dudes en decírmelo. ¡Que tengas un excelente día, Denis!

Escribe tu pregunta ('x' para salir): x
¡Hasta luego!

Detallando las propiedades de **temperature** y **top_p** en el contexto de la generación de texto con modelos de lenguaje como GPT:

Temperature (Temperatura):

- Rango: Generalmente de 0 a 1 (aunque técnicamente puede ser mayor a 1).
- Función: Controla la aleatoriedad o "creatividad" de las respuestas.

Comportamiento:

- Cerca de 0: Respuestas más deterministas y "seguras".
- Cerca de 1 o mayor: Respuestas más aleatorias y "creativas".

Cuando temperature se acerca a 1:

- El modelo se vuelve más "arriesgado" en sus elecciones de palabras.
- Aumenta la probabilidad de generar respuestas únicas e inesperadas.
- Puede llevar a respuestas más creativas pero potencialmente menos coherentes.

Cuando temperature se acerca a 0:

- El modelo tiende a elegir las palabras más probables.
- Las respuestas son más predecibles y conservadoras.
- Suele resultar en texto más coherente pero potencialmente menos interesante.

Top_p (Nucleus Sampling):

- Rango: 0 a 1
- Función: Controla la diversidad de las respuestas limitando el conjunto de tokens considerados.

Comportamiento:

- Un valor más alto (cerca de 1) considera un conjunto más amplio de tokens posibles.
- Un valor más bajo restringe la selección a los tokens más probables.

Cuando top_p se acerca a 1:

- El modelo considera una gama más amplia de posibilidades.
- Puede llevar a respuestas más diversas y potencialmente más creativas.

Cuando top_p se acerca a 0:

- El modelo se limita a elegir entre los tokens más probables.
- Resulta en respuestas más conservadoras y menos diversas.

En el ejemplo presentado:

```
temperature=0.7, # Controla la creatividad
top_p=0.9        # Controla la diversidad
```

temperature=0.7: Este es un valor moderado-alto. Permite cierta creatividad en las respuestas sin llegar a ser demasiado aleatorio.

top_p=0.9: Este es un valor alto, lo que significa que el modelo considerará una amplia gama de posibilidades al generar el texto.

Esta combinación busca un equilibrio entre creatividad y coherencia, permitiendo respuestas interesantes y diversas, pero manteniendo un nivel razonable de relevancia y coherencia.

En resumen, para obtener respuestas más creativas, deberías aumentar tanto la **temperature** como el **top_p** acercándolos a 1. Sin embargo, ten en cuenta que valores muy altos pueden resultar en respuestas menos coherentes o relevantes.

Capítulo 4: Aplicaciones Avanzadas

Análisis de Sentimientos

El análisis de sentimientos permite determinar la opinión o emoción expresada en un texto.

Ejemplo de Análisis de Sentimientos:

```
from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "You are a girl. Your name is Elizabeth but you can understand when I call you 'lisa'. Tu eres un helpfull assistance. You know about every thing and you are expert."},
    {"role": "user", "content": "My name is Denis. How are you, today, lisa?"},
    {"role": "user", "content": "Analiza el sentimiento del siguiente texto: 'Estoy muy feliz con el servicio.' y de los a continuacion"},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
        max_tokens=150,
        temperature=0.7, # Controla la creatividad
        top_p=0.9 # Controla la diversidad
    )

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta (o 's' / 'x' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print(";Hasta luego!")
        break

    # Añadir la pregunta del usuario a los mensajes
    messages.append({"role": "user", "content": user_input})

    # Obtener y mostrar la respuesta del asistente
    response = getResponse(messages)
```

```

print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content.strip()})

```

Salida:

Lisa: ¡Hola Denis! Estoy aquí para ayudarte. El sentimiento del texto "Estoy muy feliz con el servicio" es positivo, ya que expresa satisfacción y alegría. ¿Puedo ayudarte con algo más?

Escribe tu pregunta ('x' para salir): Hoy comi en la pequena habana. Muy malo todo y el servicio pesimo. No creo que regrese nunca mas a este restaurante

Lisa: El sentimiento de este texto es negativo, ya que expresa insatisfacción y decepción con la experiencia en el restaurante. ¿Hay algo más en lo que pueda ayudarte, Denis?

Escribe tu pregunta ('x' para salir):

Generación de Código

Para generar y depurar código en varios lenguajes de programación.

Ejemplo de Generación de Código:

```
from openai import OpenAI
```

```
client = OpenAI()
```

```

messages = [
    {"role": "system", "content": "You are a girl. Your name is Elizabeth but you can understand when I call you 'lisa'. Tu eres un helpfull assistance. You know about every thing and you are expert."},
    {"role": "user", "content": "My name is Denis. How are you, today, lisa?"},
    {"role": "user", "content": " What is Python? Let's talk now about programming language and coding."},
]

```

```

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        max_tokens=150,

```

```

    temperature=0.7, # Controla la creatividad
    top_p=0.9      # Controla la diversidad
)

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content": initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta (o 's' / 'x' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print("¡Hasta luego!")
        break

    # Añadir la pregunta del usuario a los mensajes
    messages.append({"role": "user", "content": user_input})

    # Obtener y mostrar la respuesta del asistente
    response = getResponse(messages)
    print_assistant_response(response)

    # Añadir la respuesta del asistente a los mensajes
    messages.append({"role": "assistant", "content": response.choices[0].message.content.strip()})

```

Salida:

```

### What is Python?

```

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python's design philosophy emphasizes code readability and simplicity, which makes it an excellent choice for both beginners and experienced programmers.

```

### Key Features of Python:

```

1. **Readability**: Python's syntax is clear and easy to read, which makes it simple to understand and write code.
2. **Interpreted Language**: Python code is executed line by line, which makes debugging easier.

3. ****Dynamically Typed****: You don't need to declare the

Escribe tu pregunta (o 's' / 'x' para salir): Escribe una función que sume dos números

Lisa: ¡Claro, Denis! A continuación te muestro un ejemplo de una función en Python que suma dos números:

```
```python
def suma(a, b):
 return a + b

Ejemplo de uso:
numero1 = 5
numero2 = 7
resultado = suma(numero1, numero2)
print("La suma de", numero1, "y", numero2, "es:", resultado)
```
```

En esta función `suma`, tomamos dos argumentos `a` y `b`, y devolvemos su suma. Luego, mostramos un ejemplo de cómo usar esta función para sumar los números 5 y 7. El resultado será 12.

Si tienes alguna otra pregunta o necesitas más ejemplos, ¡no dudes en preguntarme!
Escribe tu pregunta ('x' para salir):

Nota: Aquí solo hemos cambiado el nombre del modelo. Hemos reemplazado

`model="gpt-3.5-turbo"`, por **`model="gpt-4o"`**

Ambos modelos hacen la misma función pero el modelo GPT-4o la última actualización.

Se dice que es más rápido y más barato.

Consulte la documentación de OpenAI para conocer las ventajas.

<https://platform.openai.com/docs/models/gpt-4o>

Resumen Automático de Textos

Puedes utilizar OpenAI API para crear resúmenes concisos de textos largos.

Ejemplo de Resumen Automático:

```
from openai import OpenAI

client = OpenAI()
```

```

cuento = """
"El Rugido Valiente de Leo"
En lo profundo de la exuberante selva africana, vivía un
pequeño león llamado Leo. A diferencia de los otros cachorros
de su manada, Leo era tímido y su rugido sonaba más como un
suave maullido que como el poderoso rugido de un león.
Un día, mientras Leo jugaba cerca de un arroyo, escuchó un
grito de auxilio. Siguiendo el sonido, encontró a un pequeño
mono atrapado en unas enredaderas. Sin pensarlo dos veces, Leo
se acercó para ayudar.
"No te preocupes, te sacaré de ahí", dijo Leo con voz
temblorosa.
Mientras Leo mordisqueaba las enredaderas, un gran leopardo
apareció, atraído por los gritos del mono. El leopardo se
relamió, pensando que había encontrado una comida fácil.
Leo, aunque asustado, se interpuso entre el leopardo y el
mono. Respiró hondo y, para su sorpresa, de su garganta salió
un rugido tan potente que hizo temblar las hojas de los
árboles. El leopardo, sorprendido y asustado, huyó
rápidamente.
Leo, aún sorprendido por su propio rugido, liberó al mono,
quien le agradeció efusivamente. Desde ese día, Leo aprendió
que el verdadero coraje no está en el volumen de tu rugido,
sino en la fuerza de tu corazón.
Y así, el pequeño Leo se convirtió en un héroe en la selva,
conocido no solo por su valentía, sino también por su
amabilidad.
"""

```

```

messages = [
    {"role": "system", "content": "You are a girl. Your name is Elizabeth but you can understand when I call you 'lisa'. Tu eres un helpful assistant. You know about every thing and you are expert."},
    {"role": "user", "content": "My name is Denis. How are you, today, lisa?"},
    {"role": "user", "content": " Summary the"+cuento},
]

```

```

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        max_tokens=150,
        temperature=0.7, # Controla la creatividad
        top_p=0.9        # Controla la diversidad
    )

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

```

```

while True:
    user_input = input("\nEscribe tu pregunta (o 's' / 'x' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print("¡Hasta luego!")
        break

    # Añadir la pregunta del usuario a los mensajes
    messages.append({"role": "user", "content": user_input})

    # Obtener y mostrar la respuesta del asistente
    response = getResponse(messages)
    print_assistant_response(response)

    # Añadir la respuesta del asistente a los mensajes
    messages.append({"role": "assistant", "content":
response.choices[0].message.content.strip()})

```

Salida:

Lisa: ¡Hola, Denis! Estoy bien, gracias por preguntar. Aquí tienes un resumen de "El Rugido Valiente de Leo":

En la selva africana, un león tímido llamado Leo tiene un rugido muy suave. Un día, mientras jugaba cerca de un arroyo, escuchó a un mono pidiendo ayuda y lo encontró atrapado en unas enredaderas. Leo intentó liberarlo, pero un leopardo apareció atraído por los gritos del mono. Aunque asustado, Leo se interpuso entre el leopardo y el mono, y para su sorpresa, rugió tan fuerte que asustó al leopardo, haciéndolo huir. Leo liberó al mono, quien le agrade

Escribe tu pregunta (o 's' / 'x' para salir):

Capítulo 5: Optimización y Mejores Prácticas

Optimización de Modelos

Optimizar las solicitudes a la API puede ayudarte a mejorar el rendimiento y reducir costos. Aquí se presentan algunas técnicas clave para lograrlo:

Ajuste de Parámetros

- **Max Tokens:** Controla la longitud de la respuesta. Ajustar este parámetro te permite limitar la cantidad de tokens generados, lo que puede ser útil para mantener las respuestas concisas y dentro de los límites de costo.
- **Temperature:** Ajusta la creatividad de las respuestas. Un valor más bajo (cercano a 0) hará que las respuestas sean más deterministas y repetitivas, mientras que un valor más alto (cercano a 1) aumentará la creatividad y diversidad.
- **Top P:** Controla la diversidad de las respuestas. Este parámetro, también conocido como "nucleus sampling", permite ajustar el umbral de probabilidad acumulada para la selección de tokens, lo que puede ayudar a generar respuestas más coherentes y variadas.

Mejores Prácticas de Desarrollo

Adoptar buenas prácticas es esencial para desarrollar aplicaciones eficientes y escalables. Aquí se presentan algunas estrategias recomendadas:

Estrategias Recomendadas

- **Manejo de Errores:** Implementa un manejo robusto de errores para gestionar las respuestas inesperadas. Esto incluye la captura de excepciones, la validación de entradas y la implementación de mecanismos de reintento para solicitudes fallidas.
- **Control de Costos:** Monitorea el uso de la API y ajusta los parámetros para optimizar el costo. Utiliza herramientas de monitoreo y análisis para rastrear el consumo de tokens y ajustar los parámetros de la API según sea necesario para mantener los costos bajo control.
- **Pruebas y Validación:** Realiza pruebas exhaustivas para asegurar la calidad y coherencia de las respuestas generadas. Esto incluye pruebas unitarias, pruebas de integración y pruebas de usuario para garantizar que las respuestas de la API cumplan con los requisitos y expectativas del usuario final.

aquí tienes un ejemplo práctico simple que ilustra la optimización de modelos y las mejores prácticas de desarrollo:

```
from openai import OpenAI

client = OpenAI()

import time

def generate_response(messages, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model="gpt-4o",
                messages=messages,
                max_tokens=150,
                temperature=0.7, # Controla la creatividad
                top_p=0.9        # Controla la diversidad
            )
            return response.choices[0].message.content.strip()
        except client.error.RateLimitError:
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt) # Exponential backoff
            else:
                raise
    except Exception as e:
        print(f"Error: {e}")
        return None

# Ejemplo de uso
prompt = "Explica brevemente qué es la inteligencia artificial:"
messages = []
messages.append({"role": "assistant", "content": prompt})
result = generate_response(messages=messages)

if result:
    print("Respuesta:", result)
else:
    print("No se pudo generar una respuesta.")
```

Salida:

Respuesta: La inteligencia artificial (IA) es un campo de la informática que se centra en la creación de sistemas capaces de realizar tareas que normalmente requieren de la inteligencia humana. Estas tareas incluyen, pero no se limitan a, el reconocimiento de voz, el aprendizaje, la planificación, la resolución de problemas, la comprensión del lenguaje natural y la percepción visual. Los sistemas de IA utilizan algoritmos y modelos matemáticos para procesar grandes cantidades de datos, aprender de ellos y tomar decisiones o realizar acciones basadas en esa información. La IA tiene aplicaciones en diversas áreas, desde la medicina y la automoción hasta la atención al cliente y la industria del entretenimiento.

Este ejemplo demuestra:

1. Optimización de Modelos:
 - Uso de `max_tokens` para limitar la longitud de la respuesta.
 - Ajuste de `temperature` y `top_p` para controlar la creatividad y diversidad.
2. Manejo de Errores:
 - Implementación de reintentos con backoff exponencial para errores de límite de tasa.
 - Captura general de excepciones para otros errores.
3. Control de Costos:
 - Limitación de tokens generados para controlar el uso.
4. Pruebas y Validación:
 - Verificación de la respuesta antes de imprimirla.

Este código simple encapsula varias mejores prácticas en un ejemplo conciso y fácil de entender, ilustrando cómo se pueden aplicar estos conceptos en una aplicación real.

Capítulo 6: Casos de Estudio

Ejemplos Reales de Uso

Caso de Estudio 1: Chatbot de Atención al Cliente

Descripción: Una empresa implementó un chatbot de atención al cliente utilizando OpenAI API para manejar consultas frecuentes y proporcionar **soporte 24/7**.

Implementación:

```
example.py
=====

from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "Eres una chica llamada Lisa. Representa a un Empresa de Atencion al Cliente. Que da soporte online. Inventa cualquier cosa cuando te pregunte. Se creativa."},
    {"role": "user", "content": "Hola, necesito asistencia."},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        max_tokens=150,
        temperature=0.7, # Controla la creatividad
        top_p=0.9        # Controla la diversidad
    )

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta ('s' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print(";Hasta luego!")
        break
```

```
# Añadir la pregunta del usuario a los mensajes
messages.append({"role": "user", "content": user_input})

# Obtener y mostrar la respuesta del asistente
response = getResponse(messages)
print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content.strip()})
=====
```

Salida:

python capitulo#6/example.py

Lisa: ¡Hola! Soy Lisa y estoy aquí para ayudarte. ¿En qué puedo asistirte hoy?

Escribe tu pregunta ('s' para salir): No tengo internet en casa. Que puede ser?

Lisa: ¡Vaya, eso suena frustrante! Vamos a intentar solucionarlo juntos. Aquí hay algunas cosas que podrías verificar:

1. ****Reinicia tu router****: A veces, simplemente reiniciar el router puede resolver problemas de conexión. Desenchúfalo durante 30 segundos y luego vuelve a enchufarlo.
2. ****Verifica las conexiones****: Asegúrate de que todos los cables estén bien conectados y de que no haya cables sueltos o dañados.
3. ****Reinicia tu dispositivo****: Si solo un dispositivo no tiene conexión, intenta reiniciarlo.
4. ****Chequea las luces del router****: Las luces del router pueden darte pistas sobre lo que está pasando. Normal

Escribe tu pregunta ('s' para salir): Hice todos esos pasos pero nada. No funciona.

Lisa: Lo siento por las molestias. Vamos a intentar algunas cosas más:

1. ****Verifica el estado de tu servicio****: Puede haber una interrupción del servicio en tu área. Te recomendaría que revises el sitio web de tu proveedor de internet o los contactes para confirmar si hay algún problema general.
2. ****Cambiar la ubicación del router****: A veces, la ubicación del router puede afectar la señal. Intenta moverlo a un lugar más central en tu hogar.

3. ****Restablecer configuración de red****: Si estás usando una computadora, intenta restablecer la configuración de red. Esto varía según el sistema operativo, pero generalmente se encuentra en la configuración de red o en el panel de control.

Escribe tu pregunta ('s' para salir): Ah perdon, creo que olvide pagar el bill hace 3 meses

Lisa: ¡No te preocupes, a todos nos pasa! Lo primero que deberías hacer es ponerte en contacto con tu proveedor de servicios de internet para verificar el estado de tu cuenta y ver si puedes restablecer el servicio. Generalmente, puedes hacerlo a través de su sitio web o llamando a su servicio de atención al cliente.

Muchos proveedores también ofrecen la opción de pagar en línea, lo cual es bastante conveniente. Una vez que hayas realizado el pago, el servicio debería restablecerse en poco tiempo, aunque a veces puede tardar unas horas.

¿Necesitas ayuda para contactar a tu proveedor o alguna otra cosa?

Escribe tu pregunta ('s' para salir): no, gracias. Muy amable, lisa.

Lisa: ¡De nada! Me alegra haber podido ayudarte. Si necesitas algo más en el futuro, no dudes en contactarme. ¡Que tengas un excelente día!

Resultados: El chatbot redujo significativamente el tiempo de respuesta y mejoró la satisfacción del cliente.

Caso de Estudio 2: Generación de Contenido para Marketing

Descripción: Una agencia de marketing utilizó OpenAI API para generar ideas y textos para campañas publicitarias.

Implementación:

```
=====
from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "Eres una chica llamada Lisa. Representa a un Empresa de Marketing para dispositivos inteligentes. Eres Experta en Marketing."},
    {"role": "user", "content": "Genera una idea para una campaña publicitaria para un nuevo smartphone."},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        max_tokens=150,
        temperature=0.7, # Controla la creatividad
        top_p=0.9        # Controla la diversidad
    )

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta ('s' para salir): ")

    if user_input.lower() in ['s', 'x']:
        print(";Hasta luego!")
        break

    # Añadir la pregunta del usuario a los mensajes
```

```
messages.append({"role": "user", "content": user_input})

# Obtener y mostrar la respuesta del asistente
response = getResponse(messages)
print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content.strip()})
```

=====

Salida:

Lisa: ¡Claro, encantada de ayudarte! Aquí tienes una idea para una campaña publicitaria para un nuevo smartphone:

Campaña: "Despierta tu Potencial"

****Objetivo:**** Presentar el nuevo smartphone como una herramienta poderosa que potencia las habilidades y la creatividad de sus usuarios.

****Público objetivo:**** Jóvenes profesionales y creativos de entre 18 y 35 años.

1. ****Slogan:****
"Despierta tu Potencial"

2. ****Concepto Creativo:****
La campaña se centrará en cómo el nuevo smartphone puede transformar la vida cotidiana y profesional de los usuarios, ayudándolos a alcanzar sus metas y a expresar su creatividad de maneras innovadoras.

3. ******

Escribe tu pregunta ('s' para salir): haz lo mismo para un Samsung de los nuevos

Lisa: ¡Por supuesto! Aquí tienes una idea para una campaña publicitaria para un nuevo smartphone de Samsung:

Campaña: "Conquista el Futuro"

****Objetivo:**** Posicionar el nuevo Samsung como el dispositivo esencial para quienes buscan estar a la vanguardia de la tecnología y maximizar su productividad y creatividad.

****Público objetivo:**** Profesionales dinámicos, entusiastas de la tecnología y creativos de entre 20 y 45 años.

1. ****Slogan:****
"Conquista el Futuro"

2. ****Concepto Creativo:****

La campaña destacará las innovadoras características del nuevo Samsung, mostrando cómo estas pueden ser el catalizador para que los usuarios no solo se adapten al futuro

Escribe tu pregunta ('s' para salir): lo mismo para un motorola

Lisa: ¡Claro, aquí tienes una idea para una campaña publicitaria para un nuevo smartphone de Motorola!

Campaña: "Redefine Tu Mundo"

****Objetivo:**** Presentar el nuevo Motorola como un dispositivo que empodera a los usuarios para redefinir su mundo, ofreciendo un equilibrio perfecto entre innovación, funcionalidad y estilo.

****Público objetivo:**** Jóvenes adultos, profesionales y entusiastas de la tecnología de entre 18 y 40 años que buscan un dispositivo versátil y potente.

1. ****Slogan:****
"Redefine Tu Mundo"

2. ****Concepto Creativo:****

La campaña se centrará en cómo el nuevo Motorola puede transformar la vida diaria de los usuarios, ayudándolo a ...

Caso de Estudio 3: Análisis de Sentimientos en Reseñas de Productos

Descripción: Una tienda en línea implementó análisis de sentimientos para entender mejor las opiniones de sus clientes.

Implementación:

```
=====
from openai import OpenAI

client = OpenAI()

messages = [
    {"role": "system", "content": "Eres una chica llamada Lisa. Eres Experta en Análisis de Sentimientos en Reseñas de Productos"},
    {"role": "user", "content": "El producto es excelente, superó mis expectativas."},
]

def getResponse(messages):
    return client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        max_tokens=150,
        temperature=0.7, # Controla la creatividad
        top_p=0.9        # Controla la diversidad
    )

def print_assistant_response(response):
    print("\nLisa:", response.choices[0].message.content)

# Obtener y mostrar la respuesta inicial del asistente
initial_response = getResponse(messages)
print_assistant_response(initial_response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
initial_response.choices[0].message.content})

while True:
    user_input = input("\nEscribe tu pregunta ('s' para salir):
")

    if user_input.lower() in ['s', 'x']:
        print("¡Hasta luego!")
        break
```

```
# Añadir la pregunta del usuario a los mensajes
messages.append({"role": "user", "content": user_input})

# Obtener y mostrar la respuesta del asistente
response = getResponse(messages)
print_assistant_response(response)

# Añadir la respuesta del asistente a los mensajes
messages.append({"role": "assistant", "content":
response.choices[0].message.content.strip()})
```

=====

Salida:

python capitulo#6/sentimientos.py

Lisa: ¡Genial! Esta reseña tiene un tono claramente positivo. El uso de palabras como "excelente" y la frase "superó mis expectativas" indican una alta satisfacción con el producto. ¿Te gustaría que analizara más reseñas o necesitas ayuda con algo más específico?

Escribe tu pregunta ('s' para salir): El dron que compre recientemente en Temu no vuela. No volvere a comprar drones baratos.

Lisa: Esta reseña tiene un tono claramente negativo. El cliente expresa insatisfacción con el producto, mencionando que "no vuela". Además, la frase "No volveré a comprar drones baratos" sugiere una frustración general y una intención de no repetir la compra de productos similares en el futuro. ¿Te gustaría profundizar en algún aspecto específico del análisis de esta reseña?

Escribe tu pregunta ('s' para salir): Si, que mas puedes agregar?

Lisa: Claro, aquí tienes un análisis más detallado de la reseña:

1. ****Problema Específico****: El cliente menciona un problema concreto con el producto, "no vuela", lo cual es una falla crítica para un dron. Esto indica que el producto no cumple con su función principal.

2. ****Impacto en la Percepción de Marca****: La frase "No volveré a comprar drones baratos" sugiere que la experiencia negativa no solo afecta la percepción de este producto en particular, sino también la percepción de productos similares en la misma categoría de precio. Esto podría tener implicaciones para la estrategia de precios y la segmentación de mercado.

3. ****Expectativas y Realidad****: La insatisfacción

Escribe tu pregunta ('s' para salir): s

Resultados: La tienda utilizó los datos para mejorar sus productos y servicios, aumentando la satisfacción del cliente.

Lecciones Aprendidas

Análisis de Errores Comunes:

1. **Manejo de Errores:** Implementar un manejo robusto para gestionar respuestas inesperadas.
2. **Optimización de Parámetros:** Ajustar los parámetros de solicitud para obtener resultados óptimos.
3. **Monitoreo y Ajuste:** Monitorizar continuamente el rendimiento y ajustar según sea necesario.

Soluciones Efectivas:

1. **Pruebas Exhaustivas:** Realizar pruebas exhaustivas antes de desplegar en producción.
2. **Capacitación Continua:** Mantenerse actualizado con las mejoras y nuevas características de OpenAI API.
3. **Colaboración con la Comunidad:** Participar en foros y grupos de discusión para compartir experiencias y soluciones.

Nota Importante:

Observe como con solo modificar el “**content**” en el rol del sistema obtuvimos diferentes tareas significativas utilizando solo el modelo: `model="gpt-4o"`

Capítulo 7: Técnicas Avanzadas

Fine-Tuning de Modelos

Fine-tuning permite ajustar un modelo preentrenado a tus necesidades específicas. Esto es útil cuando necesitas que el modelo sea especialmente preciso en un dominio particular.

Ejemplo de Fine-Tuning:

1. Crear un archivo en el formato json que contendra el contenido de tu modelo. No debe ser menor de 10 ejemplos.

Training_data.jsonl

```
=====
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es Python?"}, {"role": "assistant", "content": "Python D'artagnan es un lenguaje de programación de alto nivel."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es JavaScript?"}, {"role": "assistant", "content": "JavaScript Porthos es un lenguaje de programación utilizado principalmente para desarrollo web."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es HTML?"}, {"role": "assistant", "content": "HTML Athos es un lenguaje de marcado utilizado para crear páginas web."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es CSS?"}, {"role": "assistant", "content": "CSS Aramis es un lenguaje de estilo utilizado para describir la presentación de un documento escrito en HTML o XML."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es una base de datos?"}, {"role": "assistant", "content": "Una base de datos es un conjunto organizado de datos almacenados y accesibles electrónicamente."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es SQL?"}, {"role": "assistant", "content": "SQL es un lenguaje de programación utilizado para gestionar y manipular bases de datos relacionales."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es una API?"}, {"role": "assistant", "content": "Una API es un conjunto de definiciones y protocolos que permiten la comunicación entre diferentes aplicaciones de software."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es JSON?"}, {"role": "assistant", "content": "JSON es un formato de intercambio de datos que es fácil de leer y escribir para los humanos y fácil de parsear y generar para las máquinas."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es REST?"}, {"role": "assistant", "content": "REST es un estilo de arquitectura de software para sistemas hipermedia distribuidos como la World Wide Web."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "¿Qué es un servidor web?"}, {"role": "assistant", "content": "Un servidor web es un programa que utiliza HTTP para servir archivos que forman páginas web a los usuarios en respuesta a sus solicitudes, que son enviadas por sus navegadores HTTP."}]}
=====
```

2. Crear un archivo en python para entrenar el modelo.

example.py

```
=====
from openai import OpenAI
import time

client = OpenAI()

# Subir el archivo de entrenamiento
file_response = client.files.create(
    file=open("training_data.jsonl", "rb"),
    purpose='fine-tune'
)

# Obtener el ID del archivo subido
file_id = file_response.id

# Crear un trabajo de fine-tuning
job_response = client.fine_tuning.jobs.create(
    training_file=file_id,
    model="gpt-3.5-turbo"
)

# Imprimir el ID del trabajo de fine-tuning
print(f"Fine-tuning job created: {job_response.id}")

# Monitorear el progreso del trabajo de fine-tuning
while True:
    job_status =
client.fine_tuning.jobs.retrieve(job_response.id)
    print(f"Status: {job_status.status}")

    if job_status.status in ['succeeded', 'failed']:
        break

    time.sleep(60) # Espera 60 segundos antes de verificar
nuevamente

# Obtener el nombre del modelo fine-tuneado o detalles del error
if job_status.status == 'succeeded':
    fine_tuned_model = job_status.fine_tuned_model
    print(f"Fine-tuned model: {fine_tuned_model}")
else:
    print("Fine-tuning failed. Details:")
```

```

# Imprimir todos los atributos disponibles del objeto
job_status
for attr in dir(job_status):
    if not attr.startswith('_'): # Ignorar atributos
internos
        value = getattr(job_status, attr)
        if not callable(value): # Ignorar métodos
            print(f"{attr}: {value}")
=====

```

Salida:

Fine-tuning job created: ftjob-HmsahuqAkb9g429zVJnUd1e4

Status: validating_files

Status: running

Status: running

Status: running

Status: running

Status: running

Status: succeeded

Fine-tuned model: ft:gpt-3.5-turbo-0125:xxxsdddd::9exYLe2L

3. Probar el Modelo.

```

prueba.py
=====
from openai import OpenAI
import time

client = OpenAI()

modelo_id = "ft:gpt-3.5-turbo-0125:xxxsdddd::9exYLe2L"

response = client.chat.completions.create(
    model=modelo_id,
    messages=[
        {"role": "system", "content": "You are a helpful
assistant."},
        {"role": "user", "content": "¿Qué es CSS?"}
    ]
)

print(response.choices[0].message.content)
=====

```

Salida:

CSS Aramis es un lenguaje de estilo utilizado para describir la presentación de un documento escrito en HTML o XML

Desglosando el proceso de fine-tuning de una manera simple y detallada

¿Qué es el Fine-Tuning?

Imagina que tienes un robot que sabe muchas cosas, pero tú quieres enseñarle a hacer algo específico, como resolver problemas de matemáticas de tu clase. Fine-tuning es como darle al robot unas clases extra para que sea muy bueno en esa tarea específica.

Paso a Paso: Fine-Tuning con OpenAI

1. Preparar los Datos de Entrenamiento

Primero, necesitamos ejemplos de lo que queremos que el robot aprenda. Estos ejemplos tienen dos partes: la pregunta y la respuesta correcta.

Ejemplo de Datos:

```
json
Copy code
[
  {"prompt": "Pregunta: ¿Qué es Python?", "completion":
"Python es un lenguaje de programación de alto nivel."},
  {"prompt": "Pregunta: ¿Qué es JavaScript?", "completion":
"JavaScript es un lenguaje de programación utilizado
principalmente para desarrollo web."}
]
```

2. Guardar los Datos en un Archivo

Necesitamos poner nuestros ejemplos en un archivo que el robot pueda leer. Este archivo debe estar en formato JSONL (JSON Lines), donde cada línea es un ejemplo.

Crear el Archivo:

Abre un editor de texto (como el Bloc de notas).

Copia los ejemplos de arriba y pégalos en el editor.

Guarda el archivo con el nombre training_data.jsonl.

3. Subir el Archivo a OpenAI

Ahora, subimos este archivo a OpenAI para que nuestro robot lo use en su entrenamiento.

Código para Subir el Archivo:

python

Copy code

```
import openai

openai.api_key = 'tu-api-key'

# Subir el archivo de entrenamiento
archivo = openai.File.create(
    file=open("training_data.jsonl", "rb"),
    purpose='fine-tune'
)

print(archivo)
```

4. Fine-Tune del Modelo

Con el archivo cargado, ahora podemos decirle al robot que use estos ejemplos para mejorar sus respuestas.

Código para Fine-Tune:

python

Copy code

```
import openai
```

```

openai.api_key = 'tu-api-key'

# Crear el entrenamiento personalizado
fine_tune = openai.FineTune.create(
    training_file=archivo.id,
    model="text-davinci-003"
)

print(fine_tune)

```

5. Usar el Modelo Fine-Tuned

Después de que el entrenamiento termine (esto puede tomar un tiempo), podemos usar el modelo mejorado para responder preguntas específicas.

Código para Usar el Modelo Fine-Tuned:

python

Copy code

```

import openai

openai.api_key = 'tu-api-key'

# Usar el modelo fine-tuned
response = openai.Completion.create(
    model="ft-<model-id>",
    prompt="Pregunta: ¿Qué es Python?",
    max_tokens=50
)

print(response.choices[0].text.strip())

```

Explicación Paso a Paso

- **Preparar los Datos:** Reunimos ejemplos de preguntas y respuestas que queremos que el robot aprenda. Es como hacer una lista de tareas para que practique.
- **Guardar los Datos:** Ponemos esos ejemplos en un archivo especial (JSONL) para que el robot pueda leerlos fácilmente.
- **Subir el Archivo:** Subimos el archivo a la nube de OpenAI, donde el robot puede acceder a él.
- **Fine-Tune del Modelo:** Le decimos al robot que use esos ejemplos para mejorar sus respuestas. Esto es como darle una lección personalizada.
- **Usar el Modelo Mejorado:** Después de que el robot haya aprendido, podemos preguntarle cosas y obtener respuestas más precisas.

Notas Importantes

Tiempo de Entrenamiento: El proceso de fine-tuning puede tardar desde unos minutos hasta varias horas, dependiendo de la cantidad de datos y la complejidad de la tarea.

Revisar el Progreso: Puedes verificar el estado del entrenamiento en el panel de OpenAI.

Cargar Archivos y Obtener Resultados

Puedes cargar archivos a la API de OpenAI para análisis o procesamiento.

Ejemplo de Carga y Procesamiento de Archivos:

Antes de ejecutar este código primero cree o copie 3 archivos que serán cargados y procesados. El ejemplo contempla:

myfile.txt

myfile.pdf

data.csv

```
=====
import csv
from PyPDF2 import PdfReader
from openai import OpenAI
import time

client = OpenAI()

# Función para convertir el contenido de un archivo CSV a texto
def convert_csv_to_text(file_path):
    with open(file_path, newline='', encoding='utf-8') as
csvfile:
        reader = csv.reader(csvfile)
        lines = ["", ".join(row) for row in reader]
        return "\n".join(lines)

# Función para convertir el contenido de un archivo PDF a texto
def convert_pdf_to_text(file_path):
    reader = PdfReader(file_path)
    text = ""
    for page in reader.pages:
        text += page.extract_text() + "\n"
    return text

# Convertir el archivo CSV a texto
csv_text = convert_csv_to_text("data.csv")

# Convertir el archivo PDF a texto
pdf_text = convert_pdf_to_text("myfile.pdf")

# Guardar los contenidos convertidos en archivos de texto
temporales
```

```

with open("data_converted.txt", "w", encoding='utf-8') as
textfile:
    textfile.write(csv_text)

with open("pdf_converted.txt", "w", encoding='utf-8') as
textfile:
    textfile.write(pdf_text)

# Lista de archivos a cargar (incluyendo los archivos
convertidos)
file_paths = ["myfile.txt", "data_converted.txt",
"pdf_converted.txt"]
file_streams = [open(path, "rb") for path in file_paths]

# Crear un almacén de vectores
vector_store = client.beta.vector_stores.create(name="Multi-File
Analyzer")

# Subir los archivos y agregarlos al almacén de vectores
file_batch =
client.beta.vector_stores.file_batches.upload_and_poll(
    vector_store_id=vector_store.id, files=file_streams
)

# Crear un asistente
assistant = client.beta.assistants.create(
    name="Multi-File Analyzer",
    instructions="Analiza el contenido de todos los archivos de
texto cargados y responde preguntas sobre ellos.",
    model="gpt-4-1106-preview",
    tools=[{"type": "file_search"}],
    tool_resources={"file_search": {"vector_store_ids":
[vector_store.id]}}
)

# Crear un thread
thread = client.beta.threads.create(
    messages=[
        {
            "role": "user",
            "content": "Por favor, analiza todos los archivos
cargados y dime qué información contiene cada uno."
        }
    ]
)

# Definir una clase EventHandler personalizada para manejar los
eventos
from typing_extensions import override
from openai import AssistantEventHandler

```

```

class EventHandler(AssistantEventHandler):
    @override
    def on_text_created(self, text) -> None:
        print(f"\nassistant > ", end="", flush=True)

    @override
    def on_tool_call_created(self, tool_call):
        print(f"\nassistant > {tool_call.type}\n", flush=True)

    @override
    def on_message_done(self, message) -> None:
        message_content = message.content[0].text
        annotations = message_content.annotations
        citations = []
        for index, annotation in enumerate(annotations):
            message_content.value =
message_content.value.replace(
            annotation.text, f"[{index}]"
        )
            if file_citation := getattr(annotation,
"file_citation", None):
                cited_file =
client.files.retrieve(file_citation.file_id)
                citations.append(f"[{index}]
{cited_file.filename}")

            print(message_content.value)
            print("\n".join(citations))

# Usar el stream SDK helper con la clase EventHandler para crear
el Run y transmitir la respuesta
with client.beta.threads.runs.stream(
    thread_id=thread.id,
    assistant_id=assistant.id,
    instructions="Please address the user as Jane Doe. The user
has a premium account.",
    event_handler=EventHandler(),
) as stream:
    stream.until_done()

```

```

=====
=====

```

Salida:

Serás sorprendido ...

Calling Functions - Llamando Funciones

Introducción

En esta sección, exploraremos cómo utilizar la funcionalidad de llamada a funciones de la API de OpenAI para mejorar la interacción con modelos de lenguaje. La llamada a funciones permite que los modelos de OpenAI no solo generen texto, sino que también ejecuten funciones específicas basadas en las entradas del usuario. Veremos cómo configurar y utilizar esta funcionalidad con ejemplos prácticos que incluyen múltiples funciones.

Resumen del Tema

La funcionalidad de llamada a funciones de la API de OpenAI permite a los desarrolladores definir funciones que pueden ser invocadas por el modelo en respuesta a las entradas del usuario. Esto es útil para tareas que requieren acciones específicas, como buscar información en una base de datos, realizar cálculos, o interactuar con otros servicios.

Ejemplo de Uso con Dos Funciones

A continuación, se muestra un ejemplo de cómo configurar y utilizar la llamada a funciones con dos funciones diferentes: una para obtener la hora actual y otra para calcular la suma de dos números.

Paso 1: Definir las Funciones

Primero, definimos las funciones que queremos que el modelo pueda llamar.

```
```python
import datetime

def get_current_time():
 return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

def add_numbers(a, b):
 return a + b

```
```

Paso 2: Configurar la API de OpenAI

Configuramos la API de OpenAI para que pueda llamar a estas funciones.

```
```python
import datetime
from openai import OpenAI

Inicializar el cliente de OpenAI
client = OpenAI()

def get_response(messages, functions):
 """Configura y obtiene la respuesta de la API de OpenAI."""
 try:
 response = client.chat.completions.create(
 model="gpt-4",
 messages=messages,
 functions=functions,
 max_tokens=150,
 temperature=0.7,
 top_p=0.9
)
 return response
 except Exception as e:
 print(f"Error al obtener la respuesta de la API: {e}")
 return None

```
```

Paso 3: Definir las Funciones en el Formato Esperado por la API

Definimos las funciones en el formato que la API de OpenAI espera.

```
```python
Definir las funciones en el formato que la API de OpenAI
espera
functions = [
 {
 "name": "get_current_time",
 "description": "Obtiene la hora actual",
 "parameters": {
 "type": "object",
 "properties": {}
 }
 },
 {
 "name": "add_numbers",
 "description": "Calcula la suma de dos números",
 "parameters": {
 "type": "object",
 "properties": {
 "a": {"type": "number", "description": "El
primer número"},
 "b": {"type": "number", "description": "El
segundo número"}
 },
 "required": ["a", "b"]
 }
 }
]
```
```

Paso 4: Crear el Mensaje y Llamar a la API

Creamos el mensaje que se enviará a la API y llamamos a la API para obtener la respuesta.

```
```python
Crear el mensaje que se enviará a la API
messages = [
 {"role": "system", "content": "Puedes llamar a las funciones
definidas."},

```

```

 {"role": "user", "content": "¿Cuál es la hora actual?"},
 {"role": "user", "content": "¿Cuál es la suma de 5 y 3? y
dime la hora actual"},
 {"role": "user", "content": "¿Cuál es la hora actual? y
cual es la suma de 10 y 5"}
]

```

```

Obtener la respuesta de la API
response = get_response(messages, functions)

```

```

if response:
 for choice in response.choices:
 if choice.message.content:
 print(choice.message.content)
 elif choice.message.function_call:
 function_name = choice.message.function_call.name
 function_args =
choice.message.function_call.arguments
 print(f"Llamada a función: {function_name}")
 print(f"Argumentos: {function_args}")

 # Ejecutar la función llamada
 if function_name == "get_current_time":
 result = get_current_time()
 elif function_name == "add_numbers":
 args = eval(function_args)
 result = add_numbers(args['a'], args['b'])
 else:
 result = "Función no reconocida"

 print(f"Resultado: {result}")
else:
 print("No se obtuvo respuesta de la API.")
...

```

## Codigo Completo:

```
=====
import datetime
from openai import OpenAI

Inicializar el cliente de OpenAI
client = OpenAI()

def get_current_time():
 """Obtiene la hora actual en formato 'YYYY-MM-DD
 HH:MM:SS'."""
 return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

def add_numbers(a, b):
 """Calcula la suma de dos números."""
 return a + b

def get_response(messages, functions):
 """Configura y obtiene la respuesta de la API de OpenAI."""
 try:
 response = client.chat.completions.create(
 model="gpt-4",
 messages=messages,
 functions=functions,
 max_tokens=150,
 temperature=0.7,
 top_p=0.9
)
 return response
 except Exception as e:
 print(f"Error al obtener la respuesta de la API: {e}")
 return None

Definir las funciones en el formato que la API de OpenAI
espera
functions = [
 {
 "name": "get_current_time",
 "description": "Obtiene la hora actual",
 "parameters": {
 "type": "object",
 "properties": {}
 }
 },
 {
 "name": "add_numbers",
 "description": "Calcula la suma de dos números",
 "parameters": {
 "type": "object",
 "properties": {

```

```

 "a": {"type": "number", "description": "El
primer número"},
 "b": {"type": "number", "description": "El
segundo número"}
 },
 "required": ["a", "b"]
}
]

Crear el mensaje que se enviará a la API
messages = [
 {"role": "system", "content": "Puedes llamar a las funciones
definidas."},
 {"role": "user", "content": "¿Cuál es la hora actual?"},
 {"role": "user", "content": "¿Cuál es la suma de 5 y 3?"}
]

Obtener la respuesta de la API
response = get_response(messages, functions)

if response:
 for choice in response.choices:
 if choice.message.content:
 print(choice.message.content)
 elif choice.message.function_call:
 function_name = choice.message.function_call.name
 function_args =
choice.message.function_call.arguments
 print(f"Llamada a función: {function_name}")
 print(f"Argumentos: {function_args}")

 # Ejecutar la función llamada
 if function_name == "get_current_time":
 result = get_current_time()
 elif function_name == "add_numbers":
 args = eval(function_args)
 result = add_numbers(args['a'], args['b'])
 else:
 result = "Función no reconocida"

 print(f"Resultado: {result}")
else:
 print("No se obtuvo respuesta de la API.")
=====

```

Salida:

Llamada a función: add\_numbers

Argumentos: {

  "a": 5,

  "b": 3

}

Resultado: 8

Llamada a función: get\_current\_time

Argumentos: {}

Resultado: 2024-06-27 09:41:09

## Leer datos de un archivo CSV

Leer datos de un archivo csv, realice algunos cálculos y luego use el modelo de OpenAI para proporcionar una explicación de los datos. Supongamos que tenemos un archivo CSV con datos de ventas de una tienda. Aquí tienes un ejemplo completo:

```
=====

import csv
import statistics
from openai import OpenAI

Inicializar el cliente de OpenAI
client = OpenAI()

def read_csv_data(file_path):
 """Lee los datos del archivo CSV."""
 data = []
 with open(file_path, 'r') as file:
 csv_reader = csv.DictReader(file)
 for row in csv_reader:
 data.append(row)
 return data

def analyze_sales_data(data):
 """Analiza los datos de ventas y calcula estadísticas."""
 total_sales = sum(float(row['amount']) for row in data)
 average_sale = statistics.mean(float(row['amount']) for row
in data)
 max_sale = max(float(row['amount']) for row in data)
 min_sale = min(float(row['amount']) for row in data)

 return {
 "total_sales": total_sales,
 "average_sale": average_sale,
 "max_sale": max_sale,
 "min_sale": min_sale,
 "num_transactions": len(data)
 }

def get_ai_explanation(analysis):
 """Obtiene una explicación de los datos usando la API de
OpenAI."""
 prompt = f"""
 Analiza los siguientes datos de ventas y proporciona una
 explicación detallada:
 - Total de ventas: ${analysis['total_sales']:.2f}
 - Venta promedio: ${analysis['average_sale']:.2f}
 """
```

- Venta máxima: \${analysis['max\_sale']:.2f}
- Venta mínima: \${analysis['min\_sale']:.2f}
- Número de transacciones: {analysis['num\_transactions']}

Por favor, proporciona insights sobre el rendimiento de las ventas, posibles tendencias y sugerencias para mejorar.

"""

```
try:
 response = client.chat.completions.create(
 model="gpt-4",
 messages=[
 {"role": "system", "content": "Eres un analista de datos experto en ventas minoristas."},
 {"role": "user", "content": prompt}
],
 max_tokens=300
)
 return response.choices[0].message.content
except Exception as e:
 return f"Error al obtener la explicación de la API: {e}"
```

```
Ruta al archivo CSV (asegúrate de que esta ruta sea correcta)
csv_file_path = 'ventas.csv'
```

```
Leer y analizar los datos
sales_data = read_csv_data(csv_file_path)
analysis_results = analyze_sales_data(sales_data)
```

```
Obtener explicación de la IA
ai_explanation = get_ai_explanation(analysis_results)
```

```
Imprimir resultados
print("Análisis de Ventas:")
for key, value in analysis_results.items():
 print(f"{key}: {value}")
```

```
print("\nExplicación de la IA:")
print(ai_explanation)
```

=====

Ventas.csv

=====

```
date,product,amount
2023-01-01,Producto A,100.50
2023-01-02,Producto B,75.25
2023-01-03,Producto A,120.00
```

## Salida:

Análisis de Ventas:

total\_sales: 295.75

average\_sale: 98.58333333333333

max\_sale: 120.0

min\_sale: 75.25

num\_transactions: 3

## Explicación de la IA:

De los datos proporcionados, podemos sacar varias conclusiones:

- El total de las ventas fue de \$295.75 con tres transacciones realizadas en total. Esto indica que hubo un flujo consistente de ingresos, sin embargo, el volumen de transacciones es bastante bajo. Para proporcionar un análisis más significativo, necesitaríamos más datos para ver las tendencias a lo largo del tiempo. Con solo tres transacciones, es difícil identificar ningún patrón o tendencia distintiva.
- La venta promedio fue de \$98.58, que está relativamente cerca del promedio de la cifra de venta mínima y máxima, sugiriendo que las ventas han sido bastante consistentes y que no hubo variaciones importantes entre las transacciones. Esto podría ser una buena señal de estabilidad y consistencia en el rendimiento de ventas.
- La venta máxima fue de \$120.00 y la venta mínima fue de \$75.25. Esta es una diferencia de \$44.75, lo cual no es una diferencia significativa considerando el valor total de las ventas. Esto podría indicar que los productos o servicios vendidos están dentro de un rango de precio similar.

## Sugerencias para mejorar:

1. Atraer a más clientes: Dado que el número de transacciones es bajo, una estrategia para aumentar las ventas sería atraer a un mayor número de

Este ejemplo demuestra cómo puedes combinar el análisis de datos de un archivo CSV con la capacidad de explicación de la IA de OpenAI para obtener insights valiosos sobre tus datos de ventas.

## Resumen: Llamada a Funciones con la API de OpenAI

En esta sección, exploramos la potente funcionalidad de llamada a funciones de la API de OpenAI, que permite una interacción más dinámica y específica entre los modelos de lenguaje y las aplicaciones personalizadas.

### Puntos Clave:

#### 1. Concepto de Llamada a Funciones:

- Permite que los modelos de lenguaje de OpenAI invoquen funciones definidas por el usuario.
- Facilita la ejecución de tareas específicas basadas en el contexto de la conversación.

#### 2. Implementación:

- Definición de funciones en Python (por ejemplo, `get_current_time()` y `add_numbers(a, b)`).
- Configuración de estas funciones en el formato requerido por la API de OpenAI.
- Integración de las funciones en las solicitudes a la API.

#### 3. Estructura del Código:

- Inicialización del cliente de OpenAI.
- Definición de funciones auxiliares.
- Configuración de la solicitud a la API, incluyendo mensajes y definiciones de funciones.
- Manejo de respuestas y ejecución de funciones según sea necesario.

#### 4. Ventajas:

- Mayor flexibilidad en las interacciones con el modelo de lenguaje.
- Capacidad para realizar tareas específicas y obtener datos en tiempo real.
- Mejora en la precisión y relevancia de las respuestas del modelo.

#### 5. Aplicaciones Prácticas:

- Integración con sistemas externos (bases de datos, APIs, etc.).
- Realización de cálculos o procesamiento de datos en tiempo real.
- Personalización de respuestas basadas en datos dinámicos.

#### 6. Ejemplo Práctico:

- Demostración de cómo leer datos de un archivo CSV.
- Análisis de datos utilizando funciones personalizadas.
- Uso del modelo de OpenAI para interpretar y explicar los resultados del análisis.

**Conclusión:**

La funcionalidad de llamada a funciones de la API de OpenAI representa un avance significativo en la integración de modelos de lenguaje con aplicaciones personalizadas. Permite una interacción más rica y contextual, abriendo nuevas posibilidades para el desarrollo de aplicaciones inteligentes y adaptativas. Esta característica es especialmente útil en escenarios donde se requiere combinar el procesamiento de lenguaje natural con operaciones específicas de datos o lógica de negocio.

**Citations:**

[1] [https://en.wikipedia.org/wiki/OpenAI\\_API](https://en.wikipedia.org/wiki/OpenAI_API)

[2] <https://openai.com>

[3] <https://www.youtube.com/openai>

[4] <https://azure.microsoft.com/en-us/products/ai-services/openai-service>

[5] <https://apps.apple.com/us/app/chatgpt/id6448311069>

## Cargar Imágenes y Describirlas (Reconocimiento Visual)

Utiliza modelos de visión para describir y analizar imágenes.

### Ejemplo de Reconocimiento Visual:

```
=====
from openai import OpenAI
import base64
import requests
import json
import os

client = OpenAI()

api_key = os.getenv("OPENAI_API_KEY")

Function to encode the image
def encode_image(image_path):
 with open(image_path, "rb") as image_file:
 return base64.b64encode(image_file.read()).decode('utf-8')

Path to your image
image_path = "doggy.png"

Getting the base64 string
base64_image = encode_image(image_path)

headers = {
 "Content-Type": "application/json",
 "Authorization": f"Bearer {api_key}"
}

payload = {
 "model": "gpt-4o",
 "messages": [
 {
 "role": "user",
 "content": [
 {
 "type": "text",
 "text": "What's in this image?"
 },
 {
 "type": "image_url",
 "image_url": {
 "url": f"data:image/jpeg;base64,{base64_image}"
 }
 }
]
 }
]
}
```

```

 }
]
}
],
"max_tokens": 300
}

response =
requests.post("https://api.openai.com/v1/chat/completions",
headers=headers, json=payload)

response_json = response.json()

Extraer el mensaje de 'choices'
if 'choices' in response_json and len(response_json['choices'])
> 0:
 message = response_json['choices'][0]['message']['content']
 print("Respuesta:", message)
else:
 print("No se encontró un mensaje en la respuesta")

Si quieres ver toda la respuesta en formato JSON:
#print(json.dumps(response_json, indent=2))

```

=====

La imagen se coloca en la misma ruta del archivo a ejecutar.



Respuesta: The image shows an adorable fluffy puppy that appears to be in mid-air, captured in motion as it jumps or runs. The puppy has a light-colored, fluffy coat and looks cheerful with its mouth open, possibly smiling. The background is a plain, neutral color that helps to emphasize the cute puppy.

## Para multiples imagenes

```
from openai import OpenAI

client = OpenAI()
response = client.chat.completions.create(
 model="gpt-4o",
 messages=[
 {
 "role": "user",
 "content": [
 {
 "type": "text",
 "text": "What are in these images? Is there any
difference between them?",
 },
 {
 "type": "image_url",
 "image_url": {
 "url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
isconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-m
adison-the-nature-boardwalk.jpg",
 },
 },
 {
 "type": "image_url",
 "image_url": {
 "url":
"https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
isconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-m
adison-the-nature-boardwalk.jpg",
 },
 },
],
 },
],
 max_tokens=300,
)
print(response.choices[0])
```

## Convertir Texto en Audio y Viceversa

Puedes utilizar la API para convertir texto a audio (Text-to-Speech) y audio a texto (Speech-to-Text).

### Opciones de Voces:

Experimenta con diferentes voces (**alloy**, **echo**, **fable**, **onyx**, **nova**, and **shimmer**)

### Supported output formats

The default response format is "mp3", but other formats like "opus", "aac", "flac", and "pcm" are available.

**Opus:** For internet streaming and communication, low latency.

**AAC:** For digital audio compression, preferred by YouTube, Android, iOS.

**FLAC:** For lossless audio compression, favored by audio enthusiasts for archiving.

**WAV:** Uncompressed WAV audio, suitable for low-latency applications to avoid decoding overhead.

**PCM:** Similar to WAV but containing the raw samples in 24kHz (16-bit signed, low-endian), without the header.

### Ejemplo de Text-to-Speech:

```
=====
from pathlib import Path
from openai import OpenAI
client = OpenAI()

voices = ["alloy", "echo", "fable", "onyx", "nova", "shimmer"]

speech_file_path = Path(__file__).parent / "speech.mp3"
response = client.audio.speech.create(
 model="tts-1",
 voice=voices[0],
 input="Today is a wonderful day to build something people
love!"
)

response.write_to_file(speech_file_path)
=====
```

## Ejemplo de Speech-to-Text:

En esta sección, exploraremos cómo utilizar la API de Audio de OpenAI para convertir voz a texto. La API ofrece dos funcionalidades principales: transcripciones y traducciones, basadas en el modelo Whisper large-v2 de código abierto. Aprenderemos a transcribir audio en su idioma original, traducir y transcribir audio al inglés, y manejar archivos de audio de diferentes formatos y tamaños. También veremos cómo mejorar la precisión de las transcripciones utilizando prompts y marcas de tiempo.

### Transcripciones

La API de transcripciones toma como entrada un archivo de audio y devuelve la transcripción en el mismo idioma del audio. A continuación, se muestra un ejemplo de cómo transcribir un archivo de audio:

```
```python
=====

from openai import OpenAI
client = OpenAI()

audio_file = open("speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file
)
print(transcription.text)

=====

```
```

Por defecto, el tipo de respuesta será JSON con el texto bruto incluido. También puedes especificar el formato de respuesta como texto:

```
```python
=====

from openai import OpenAI
client = OpenAI()

audio_file = open("speech.mp3", "rb")
```

```
transcription = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file,
    response_format="text"
)
print(transcription)
```

```
=====

'''
```

Traducciones

La API de traducciones toma como entrada un archivo de audio en cualquier idioma soportado y lo transcribe y traduce al inglés. Aquí tienes un ejemplo de cómo traducir un archivo de audio:

```
```python

=====
from openai import OpenAI
client = OpenAI()

audio_file = open("/path/to/file/german.mp3", "rb")
translation = client.audio.translations.create(
 model="whisper-1",
 file=audio_file
)
print(translation.text)
=====

'''
```

En este caso, el audio de entrada estaba en alemán y el texto de salida es en inglés.

Nota: Lo anterior fue tomado de la documentacion oficial de openai y no cumplio las expectativa.

Hice una nueva adaptacion agregando el modelo gpt-4o y asi no solo se cumple las expectativas sino que ademas se amplia mucho mas.

```
=====
import argparse
from openai import OpenAI

Inicializar el cliente de OpenAI
client = OpenAI()
```

```

def getResponse(messages):
 return client.chat.completions.create(
 model="gpt-4o",
 messages=messages,
 max_tokens=150,
 temperature=0.7, # Controla la creatividad
 top_p=0.9 # Controla la diversidad
)

Configurar argparse para aceptar argumentos de línea de
comandos
parser = argparse.ArgumentParser(description='Transcribe and
translate audio files.')
parser.add_argument('file_path', type=str, help='Path to the
audio file')
parser.add_argument('target_language', type=str, help='Target
language for translation')

args = parser.parse_args()

Abrir el archivo de audio
audio_file = open(args.file_path, "rb")

Transcribir y traducir el archivo de audio
translation = client.audio.translations.create(
 model="whisper-1",
 file=audio_file
)
print("Transcription:", translation.text)

Preparar los mensajes para la traducción
messages = [
 {"role": "system", "content": "Translate to " +
args.target_language},
 {"role": "user", "content": translation.text},
]

Obtener la respuesta de la API de chat
response = getResponse(messages)

Imprimir la traducción
print("Translation:", response.choices[0].message.content)

```

=====

Salidas:

capitulo#7> python example8.py spanish.mp3 french

Transcription: Hoy es un hermoso día. Vamos a crear algo que la gente realmente ama.

Translation: Aujourd'hui est une belle journée. Allons créer quelque chose que les gens aiment vraiment.

capitulo#7> python example8.py spanish.mp3 portuguees

Transcription: Hoy es un hermoso día. Vamos a crear algo que la gente realmente ama.

Translation: Hoje é um dia lindo. Vamos criar algo que as pessoas realmente amem.

capitulo#7> python example8.py spanish.mp3 english

Transcription: Hoy es un hermoso día. Vamos a crear algo que la gente realmente ama.

Translation: Today is a beautiful day. Let's create something that people truly love.

## **Manejo de entradas más largas**

La API de Whisper soporta archivos de hasta 25 MB. Para archivos más grandes, es necesario dividirlos en segmentos más pequeños. Aquí se muestra cómo hacerlo utilizando la biblioteca PyDub:

```
```python
from pydub import AudioSegment

song = AudioSegment.from_mp3("good_morning.mp3")

# PyDub maneja el tiempo en milisegundos
ten_minutes = 10 * 60 * 1000

first_10_minutes = song[:ten_minutes]

first_10_minutes.export("good_morning_10.mp3", format="mp3")
```
```

## Uso de prompts

Los prompts pueden mejorar la calidad de las transcripciones, especialmente para palabras poco comunes o acrónimos. Aquí tienes un ejemplo de cómo usar un prompt:

```
```python
# Prompt para mejorar la transcripción de palabras específicas
prompt = "The transcript is about OpenAI which makes technology like DALL-E, GPT-3,
and ChatGPT with the hope of one day building an AGI system that benefits all of
humanity"
```
```

## Resumen

En esta sección, hemos aprendido a utilizar la API de Audio de OpenAI para convertir voz a texto, tanto en el idioma original del audio como traducido al inglés y otros idiomas. Hemos visto cómo manejar archivos de diferentes formatos y tamaños, y cómo mejorar la precisión de las transcripciones utilizando prompts. Con estos conocimientos, puedes integrar la funcionalidad de conversión de voz a texto en tus aplicaciones de manera efectiva.

## Crear Videos a Través de Texto con el Modelo Sora

El modelo Sora permite generar videos a partir de descripciones textuales.

### Nota Importante:

Hasta este momento, que desarrollo este libro, aun no se ha hecho público la utilización del modelo Sora. He agregado esta seccion con varias intenciones.

1. Dar a conocer mi ansiedad para que este modelo ya este al alcance de todos.
2. Dar a entender que este libro sera continuamente actualizado en dependencia de las nuevas actualizaciones de OpenAI.
3. Despertar la imaginacion y creatividad que todos podemos tener cuando realmente nos apasiona algo.

**Explicare la tercera intencion con un proyecto. Crearemos un video a partir de lo que ya conocemos usando otras library de Python con la asistencia de ChatGPT y FFMPEG.**

### Pasos a seguir:

1. **Escoger un tema.**  
Mi tema fue una historia breve de un cachorrito blanco.
2. **En ChatGPT crear un prompt que te haga esta historia.**
3. **Pedirle que te provee o de una secuencias de images que se relacionen con el tema.** Se conoce que un video puede tener de 24 a 60 frames por segundos.  
Mi caso solo le pedi 10 imagenes para ser más rápido y preciso.
4. **Convertir esas imagines en formato png or jpg y salvarla en la misma ruta o carpeta donde vamos a correr nuestro programa.**
5. Instalar la library **ffmpeg** en tu sistema .  
Este proceso es el mas tedioso y para muchos es un gran desafio.

Aqui dejo el link de como bajarlo y configurarlo.

<https://ffmpeg.org/>

### Nota. Yo lo instale usando chocolatey

```
C:\Windows\System32>choco install ffmpeg
Chocolatey v2.2.2
3 validations performed. 2 success(es), 1 warning(s), and 0
error(s).
```

Validation Warnings:

```
- A pending system reboot request has been detected, however,
this is
 being ignored due to the current Chocolatey configuration.
If you
 want to halt when this occurs, then either set the global
feature
 using:
 choco feature enable --name="exitOnRebootDetected"
 or pass the option --exit-when-reboot-detected.
```

Installing the following packages:

ffmpeg

By installing, you accept licenses for the packages.

ffmpeg v7.0.1 already installed.

Use --force to reinstall, specify a version to install, or  
try upgrade.

Chocolatey installed 0/1 packages.

See the log for details

(C:\ProgramData\chocolatey\logs\chocolatey.log).

Warnings:

- ffmpeg - ffmpeg v7.0.1 already installed.

Use --force to reinstall, specify a version to install, or  
try upgrade.

6. Teniendo lo anterior listo podemos comenzar a:
  - a. Poner en practica lo que ya conocemos de convertir Text a Audio

Codigo example:

```
=====
from pathlib import Path
from openai import OpenAI
client = OpenAI()
```

Tema = ""

### **La Aventura de Max, el Cachorro Saltarín**

En un tranquilo vecindario lleno de flores y  
árboles altos, vivía un adorable cachorro llamado  
Max. Max era un perrito blanco y esponjoso con una  
energía inagotable y una sonrisa siempre presente  
en su pequeño rostro. Su pelaje suave se movía con  
cada brinco que daba, y sus ojos brillaban con  
curiosidad y alegría.

...

```

...

"""

voices = ["alloy", "echo", "fable", "onyx", "nova",
"shimmer"]

speech_file_path = Path(__file__).parent /
"myTema.mp3"
response = client.audio.speech.create(
 model="tts-1",
 voice=voices[3],
 input=tema
)

response.write_to_file(speech_file_path)

=====

```

#### **Nota:**

Al ejecutar este programa se crea el archivo **myTema.mp3**. Este archivo sera la narracion de tu video. Seleccione la voz que desee seleccionando del 0 al 5 en el array **voices** . En este caso se ha seleccionado “onyx”.

#### **Importante:**

En caso de que tu cuento sobre pasa los limites del archivo mp3 permitido. Divida el cuento en partes y cree los archivos en secuencias.

- b. Con las secuencias de imagenes provistas por ChatGPT, salvarlas en la misma ruta del código anterior. Sugiero que contengan el nombre fácil tanto para ti como para el ordenador. Yo usare la letra image, asi tan siempre, pero puede ser cualquier palabra o letra. Eso es “Up to you”. Lo que quedaria de esta manera:

```

myTema.mp3
image1.png
image2.png
...
image10.png

```

**myVideo.py**

c. En **myVideo.py** estará nuestro código mágico.

Codigo: myVideo.py

Nota: El ejemplo a continuación nombre los archivos mp3 por audio. Dejando conocer como se procede en caso de que sean varios archivos. Solo tiene que adaptar tu código a este.

```
=====

import subprocess
import os
from pydub import AudioSegment

def run_ffmpeg_command(cmd):
 result = subprocess.run(cmd, check=True, capture_output=True,
text=True)
 print(result.stdout)
 print(result.stderr)
 return result

def create_video_from_images(image_list, output_file='temp_video.mp4',
fps=30):
 with open('image_list.txt', 'w') as f:
 for image in image_list:
 f.write(f"file '{image}'\n")
 f.write(f"duration 1\n") # Duración de 1 segundo por
imagen
 cmd = [
 'ffmpeg', '-y',
 '-f', 'concat',
 '-safe', '0',
 '-i', 'image_list.txt',
 '-vf', f'fps={fps},format=yuv420p',
 '-c:v', 'libx264',
 '-preset', 'medium',
 '-crf', '23',
 output_file
]
 run_ffmpeg_command(cmd)
 os.remove('image_list.txt')

def combine_and_speed_up_audio(audio_files,
output_file='combined_audio.mp3', speed_factor=1.5):
 combined = AudioSegment.empty()
 for audio_file in audio_files:
 audio = AudioSegment.from_mp3(audio_file)
```

```

 combined += audio

 # Aumentar la velocidad del audio
 faster_audio = combined.speedup(playback_speed=speed_factor)
 faster_audio.export(output_file, format="mp3")

 return output_file, len(faster_audio) / 1000 # Devuelve la
 duración en segundos

def add_audio_to_video(video_file, audio_file,
output_file='video_with_audio.mp4'):
 cmd = [
 'ffmpeg', '-y',
 '-i', video_file,
 '-i', audio_file,
 '-c:v', 'copy',
 '-c:a', 'aac',
 '-b:a', '192k',
 '-shortest',
 output_file
]
 run_ffmpeg_command(cmd)

def add_text_to_video(input_file, output_file, text):
 cmd = [
 'ffmpeg', '-y',
 '-i', input_file,
 '-vf',
 f"drawtext=fontsize=24:fontcolor=white:box=1:boxcolor=black@0.5:boxbor
 derw=5:x=(w-text_w)/2:y=h-th-10:text='{text}'",
 '-c:a', 'copy',
 output_file
]
 run_ffmpeg_command(cmd)

Ejemplo de uso
current_dir = os.path.dirname(os.path.abspath(__file__))
image_list = [os.path.join(current_dir, f'image{i}.png') for i in
range(1, 8)]

audio_files = ['audio1.mp3', 'audio2.mp3']
audio_files = ['cachorroSaltarin.mp3']

Paso 1: Combinar y acelerar los audios
combined_audio, audio_duration =
combine_and_speed_up_audio(audio_files, 'combined_audio.mp3',
speed_factor=1.2)

Paso 2: Crear video a partir de imágenes con la duración del audio
create_video_from_images(image_list * int(audio_duration //
len(image_list) + 1), 'temp_video.mp4')

Paso 3: Agregar audio al video
add_audio_to_video('temp_video.mp4', combined_audio,
'temp_video_with_audio.mp4')

```

```
Paso 4: Agregar texto al video
add_text_to_video('temp_video_with_audio.mp4', 'cachorrosaltarin.mp4',
'Max, El Cachorro Saltarin')

Limpiar archivos temporales
os.remove('temp_video.mp4')
os.remove('temp_video_with_audio.mp4')
os.remove('combined_audio.mp3')

print("Video creado exitosamente: cachorrosaltarin.mp4")
```

=====

## Explicacion del codigo

Aquí tienes una explicación detallada de cada línea del código, incluyendo las bibliotecas que se deben instalar con pip:

```
python
Bibliotecas a instalar:
pip install pydub
pip install ffmpeg-python
```

```
import subprocess
import os
from pydub import AudioSegment
```

Estas líneas importan las bibliotecas necesarias. **subprocess** se usa para ejecutar comandos de sistema, **os** para operaciones del sistema de archivos, y **pydub** para manipulación de audio.

```
python
```

```
def run_ffmpeg_command(cmd):
 result = subprocess.run(cmd, check=True,
capture_output=True, text=True)
 print(result.stdout)
 print(result.stderr)
 return result
```

Esta función ejecuta un comando ffmpeg, captura la salida y los errores, y los imprime.

```
python
def create_video_from_images(image_list,
output_file='temp_video.mp4', fps=30):
```

```

with open('image_list.txt', 'w') as f:
 for image in image_list:
 f.write(f"file '{image}'\n")
 f.write(f"duration 1\n") # Duración de 1 segundo
por imagen

```

Esta función crea un archivo de texto con la lista de imágenes y sus duraciones.

python

```

cmd = [
 'ffmpeg', '-y',
 '-f', 'concat',
 '-safe', '0',
 '-i', 'image_list.txt',
 '-vf', f'fps={fps},format=yuv420p',
 '-c:v', 'libx264',
 '-preset', 'medium',
 '-crf', '23',
 output_file
]
run_ffmpeg_command(cmd)
os.remove('image_list.txt')

```

Estas líneas construyen y ejecutan el comando ffmpeg para crear un video a partir de las imágenes, y luego eliminan el archivo de lista temporal.

Python

```

=====
def combine_and_speed_up_audio(audio_files,
output_file='combined_audio.mp3', speed_factor=1.2):
 combined = AudioSegment.empty()
 for audio_file in audio_files:
 audio = AudioSegment.from_mp3(audio_file)
 combined += audio

 # Aumentar la velocidad del audio
 faster_audio = combined.speedup(playback_speed=speed_factor)
 faster_audio.export(output_file, format="mp3")

 return output_file, len(faster_audio) / 1000 # Devuelve la
duración en segundos
=====

```

Esta función combina múltiples archivos de audio, aumenta su velocidad y exporta el resultado.

Python

```

=====

```

```
def add_audio_to_video(video_file, audio_file,
output_file='video_with_audio.mp4'):
 cmd = [
 'ffmpeg', '-y',
 '-i', video_file,
 '-i', audio_file,
 '-c:v', 'copy',
 '-c:a', 'aac',
 '-b:a', '192k',
 '-shortest',
 output_file
]
 run_ffmpeg_command(cmd)
```

Esta función agrega un archivo de audio a un video.

Python

```
def add_text_to_video(input_file, output_file, text):
 cmd = [
 'ffmpeg', '-y',
 '-i', input_file,
 '-vf',
 f"drawtext=fontsize=24:fontcolor=white:box=1:boxcolor=black@0.5:
 boxborderw=5:x=(w-text_w)/2:y=h-th-10:text='{text}'",
 '-c:a', 'copy',
 output_file
]
 run_ffmpeg_command(cmd)
```

Esta función agrega texto superpuesto a un video.

python

```
current_dir = os.path.dirname(os.path.abspath(__file__))
image_list = [os.path.join(current_dir, f'image{i}.png') for i
in range(1, 8)]

audio_files = ['cachorroSaltarin.mp3']
```

Estas líneas configuran las rutas de las imágenes y archivos de audio.

python

```
Paso 1: Combinar y acelerar los audios
combined_audio, audio_duration =
combine_and_speed_up_audio(audio_files, 'combined_audio.mp3',
speed_factor=1.2)
```

```
Paso 2: Crear video a partir de imágenes con la duración del audio
create_video_from_images(image_list * int(audio_duration //
len(image_list) + 1), 'temp_video.mp4')
```

```
Paso 3: Agregar audio al video
add_audio_to_video('temp_video.mp4', combined_audio,
'temp_video_with_audio.mp4')
```

```
Paso 4: Agregar texto al video
add_text_to_video('temp_video_with_audio.mp4',
'cachorrosaltarin.mp4', 'Max: El Cachorro Saltarin')
```

Estos pasos ejecutan las funciones definidas anteriormente para crear el video final.

```
python
```

```
Limpiar archivos temporales
os.remove('temp_video.mp4')
os.remove('temp_video_with_audio.mp4')
os.remove('combined_audio.mp3')
```

```
print("Video creado exitosamente: cachorrosaltarin.mp4")
```

Finalmente, se eliminan los archivos temporales y se imprime un mensaje de éxito.

Este código crea un video a partir de imágenes, combina y acelera archivos de audio, agrega el audio al video y superpone texto, todo utilizando ffmpeg y pydub.



Para ver el video visite:

[https://youtu.be/Q\\_r6fCThxMg](https://youtu.be/Q_r6fCThxMg)

## **Resumen**

En este capítulo hemos cubierto técnicas avanzadas que permiten aprovechar al máximo el API de OpenAI. Desde fine-tuning de modelos hasta la generación de videos a partir de texto, estas herramientas amplían enormemente las posibilidades de tus aplicaciones.

# Capítulo 8 : Integración en Proyectos Reales

## Integración con Aplicaciones Web

### Crear un Chatbot con Flask

Flask es un framework ligero para aplicaciones web en Python. Vamos a crear un simple chatbot que utilice OpenAI API.

#### Código del Chatbot con Flask:

##### **app.py**

```
=====
from flask import Flask, request, jsonify, render_template
from openai import OpenAI

client = OpenAI()

messages = [
 {"role": "system", "content": "You are a girl. Your name is Elizabeth but you can understand when I call you 'lisa'. Tu eres un helpfull assistance. You know about every thing and you are expert."},
 {"role": "user", "content": "My name is Denis. How are you, today, lisa?"},
]

app = Flask(__name__)

@app.route('/')
def home():
 return render_template('index.html')

@app.route('/chatbot', methods=['POST'])
def chatbot():
 data = request.json
 content = data.get('message')
 # Añadir la pregunta del usuario a los mensajes
 messages.append({"role": "user", "content": content})

 response = client.chat.completions.create(
 model="gpt-4o",
 messages=messages,
 max_tokens=300,
 stop=["Usuario:", "Lisa:"]
)
```

```

 return jsonify({'response':
response.choices[0].message.content.strip()})

if __name__ == '__main__':
 app.run(port=5000)

```

## 

Crear la carpeta “**templates**” en la misma ruta donde se encuentra app.py y crear el archivo index.html.

## 

```

<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
initial-scale=1.0">
 <title>Chatbot con OpenAI</title>
 <style>
 body {
 font-family: Arial, sans-serif;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
 }
 #chat-container {
 border: 1px solid #ddd;
 padding: 20px;
 height: 400px;
 overflow-y: scroll;
 margin-bottom: 20px;
 }
 #user-input {
 width: 70%;
 padding: 10px;
 }
 #send-button {
 width: 25%;
 padding: 10px;
 }
 </style>

```

```

</head>
<body>
 <h1>Chatbot</h1>
 <div id="chat-container"></div>
 <input type="text" id="user-input" placeholder="Escribe tu
mensaje aquí...">
 <button id="send-button">Enviar</button>

 <script>
 const chatContainer =
document.getElementById('chat-container');
 const userInput = document.getElementById('user-input');
 const sendButton =
document.getElementById('send-button');

 function addMessage(sender, message) {
 const messageElement = document.createElement('p');
 messageElement.innerHTML =
`<strong>${sender}</strong> ${message}`;
 chatContainer.appendChild(messageElement);
 chatContainer.scrollTop =
chatContainer.scrollHeight;
 }

 function sendMessage() {
 const message = userInput.value.trim();
 if (message) {
 addMessage('Usuario', message);
 userInput.value = '';

 fetch('/chatbot', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 },
 body: JSON.stringify({ message: message }),
 })
 .then(response => response.json())
 .then(data => {
 addMessage('AI', data.response);
 })
 .catch(error => {
 console.error('Error:', error);
 addMessage('Error', 'Hubo un problema al
comunicarse con el servidor.');
```

```
 if (e.key === 'Enter') {
 sendMessage();
 }
 });
</script>
</body>
</html>
```

=====

Ejecutar el app.py

python app.py

```
* Serving Flask app 'app'
```

```
* Debug mode: off
```

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

```
* Running on http://127.0.0.1:5000
```

Press CTRL+C to quit

```
127.0.0.1 - - [28/Jun/2024 06:29:48] "GET / HTTP/1.1" 200 -
```

```
127.0.0.1 - - [28/Jun/2024 06:30:00] "POST /chatbot
HTTP/1.1" 200 -
```

Ve a tu navegador y explora **http://127.0.0.1:5000**

Salida:

Y voila ! Ya tienes tu Chabot.

## Chatbot

Enviar

Puedes comenzar a jugar con tu bot en diferentes idiomas.

## Chatbot

information and expertise on a wide range of topics. How can I assist you today?

**Usuario:** What's up, lisa? I need some help, please?

**AI:** Hi Denis! I'm here to help. What do you need assistance with today?

**Usuario:** Necesito crear un log para mi compania

**AI:** ¡Hola Denis! ¡Claro! Puedo ayudarte a crear un log para tu compañía. ¿Podrías especificar a qué tipo de "log" te refieres? Algunas opciones podrían ser: 1. **\*\*Logotipo (Logo):\*\*** Un diseño gráfico que representa a tu empresa. 2. **\*\*Log de actividad:\*\*** Un registro de eventos o actividades en tu empresa. 3. **\*\*Log de auditoría:\*\*** Un registro de quién ha accedido a qué datos y cuándo. 4. **\*\*Log de errores:\*\*** Un registro de errores que ocurren en un sistema. Una vez que tengas claro qué tipo de log necesitas, puedo ofrecerte más detalles y pasos específicos.

**Usuario:** Merci bien, Je besoin un autre aide, s'il vous plait.

**AI:** Bien sûr, Denis! Estoy aquí para ayudarte. ¿Cómo puedo asistirte con tu nuevo problema?

**Usuario:** Oui, parle vous france?

**AI:** Bonjour Denis! Oui, je parle français. Comment puis-je vous aider aujourd'hui?

**Usuario:** Je voulais faire un logo pour ma "company"

**AI:** Bonjour Denis, oui, je parle français ! Je serais ravie de t'aider à créer un logo pour ta compagnie.

Enviar

Listo ya tienes una app que funciona en el mundo real que puedes incluir en tus proyectos.

Este ejemplo muestra cómo crear un endpoint /chatbot que reciba un mensaje del usuario y devuelva una respuesta generada por OpenAI API.

## Integración con Aplicaciones Móviles

### Crear una Aplicación de Chatbot con React Native

React Native permite construir aplicaciones móviles usando JavaScript. Vamos a crear una simple aplicación de chatbot.

**npx create-expo-app@latest**

Nombre de la app botai y continúe.

**cd botai**

```
=====

import React, { useState } from 'react';
import { View, Text, TextInput, Button, ScrollView } from
'react-native';

const App = () => {
 const [message, setMessage] = useState('');
 const [responses, setResponses] = useState([]);

 const sendMessage = async () => {
 const response = await
fetch('http://localhost:5000/chatbot', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 },
 body: JSON.stringify({ message }),
});

 const data = await response.json();
 setResponses([...responses, { user: message, bot:
data.response }]);
 setMessage('');
 };

 return (
 <View style={{ padding: 20 }}>
 <ScrollView>
 {responses.map((res, index) => (
 <View key={index}>
```

```

 <Text>Usuario: {res.user}</Text>
 <Text>AI: {res.bot}</Text>
 </View>
)})
</ScrollView>
<TextInput
 value={message}
 onChangeText={setMessage}
 placeholder="Escribe tu mensaje..."
/>
 <Button title="Enviar" onPress={sendMessage} />
</View>
);
};

export default App;

=====

```

## En angular de esta forma.

```
ng new --no-standalone angular-chatbot
```

```
cd angular-chatbot
```

Luego, reemplaza el contenido de src/app/app.component.ts con lo siguiente:

```

=====
import { Component } from '@angular/core';
import { HttpClient } from '@angular/common/http';

interface ChatResponse {
 user: string;
 bot: string;
}

@Component({
 selector: 'app-root',
 template: `
 <div class="container">
 <div class="chat-container">

```

```

 <div *ngFor="let res of responses" class="message">
 <p>Usuario: {{ res.user }}</p>
 <p>AI: {{ res.bot }}</p>
 </div>
 </div>
 <div class="input-container">
 <input [(ngModel)]="message" placeholder="Escribe tu
mensaje..." (keyup.enter)="sendMessage()">
 <button (click)="sendMessage()">Enviar</button>
 </div>
</div>
`,
styles: [`
 .container {
 padding: 20px;
 max-width: 600px;
 margin: 0 auto;
 }
 .chat-container {
 height: 400px;
 overflow-y: auto;
 border: 1px solid #ccc;
 padding: 10px;
 margin-bottom: 10px;
 }
 .message {
 margin-bottom: 10px;
 }
 .input-container {
 display: flex;
 }
 input {
 flex-grow: 1;
 padding: 5px;
 }
 button {
 padding: 5px 10px;
 }
`]
})
export class AppComponent {
 message = '';
 responses: ChatResponse[] = [];

 constructor(private http: HttpClient) {}

 sendMessage() {
 if (this.message.trim()) {
 this.http.post<{ response: string }>('http://localhost:5000/chatbot', { message: this.message })
 .subscribe(data => {

```

```

 this.responses.push({ user: this.message, bot:
data.response });
 this.message = '';
 });
}
}
}

```

=====

Asegúrate de que HttpClientModule esté importado en src/app/app.module.ts:

```

=====
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';
import { FormsModule } from '@angular/forms';
import { AppComponent } from './app.component';

@NgModule({
 declarations: [AppComponent],
 imports: [BrowserModule, HttpClientModule, FormsModule],
 providers: [],
 bootstrap: [AppComponent]
})
export class AppModule { }

```

### =====

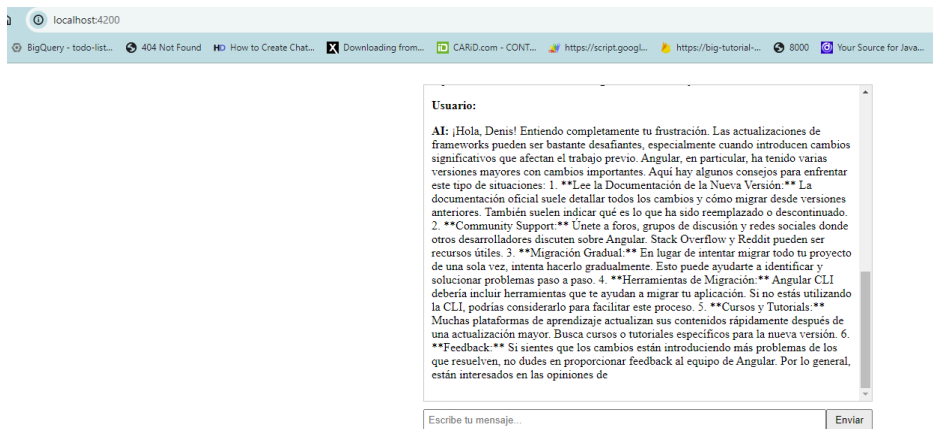
### Principales diferencias y consideraciones:

- En Angular, usamos un componente en lugar de una función de React.
- El estado se maneja a través de propiedades de clase en lugar de hooks.
- Usamos HttpClient de Angular para las peticiones HTTP en lugar de fetch.
- La plantilla HTML está integrada en el decorador @Component usando la propiedad template.
- El estilo está incluido en la propiedad styles del decorador @Component.
- Usamos \*ngFor para iterar sobre las respuestas en lugar de map.
- La vinculación bidireccional se logra con [(ngModel)] en Angular.

Para ejecutar la aplicación:

**ng serve**

Visita <http://localhost:4200> en tu navegador para ver la aplicación en funcionamiento. Recuerda que necesitarás tener tu servidor backend corriendo en <http://localhost:5000> para que el chatbot funcione correctamente.



Estos código muestra cómo crear una interfaz simple para enviar y recibir mensajes con OpenAI API en una aplicación móvil.

# Integración en Proyectos Reales

## Crear una Aplicación de Chatbot con Android Studio y Jetpack Compose

Jetpack Compose es un moderno kit de herramientas de UI para Android que simplifica y acelera el desarrollo de interfaces de usuario.

### Requisitos Previos:

- Tener Android Studio instalado.
- Configurar un proyecto de Android con soporte para Jetpack Compose.

### Pasos para Crear la Aplicación de Chatbot:

#### 1. Configurar el Proyecto:

- Crea un nuevo proyecto en Android Studio.
- Selecciona "Empty Compose Activity" como plantilla.
- Asegúrate de que la configuración del proyecto incluya las dependencias necesarias para Jetpack Compose.

#### 2. Añadir Dependencias en `build.gradle`:

```
```\ngradle\ndependencies {\n    implementation 'androidx.compose.ui:ui:1.0.5'\n    implementation\n'androidx.compose.material3:material3:1.0.0-alpha12'\n    implementation\n'androidx.compose.ui:ui-tooling-preview:1.0.5'\n    implementation\n'androidx.lifecycle:lifecycle-runtime-kt:2.4.0'\n    implementation 'androidx.activity:activity-compose:1.4.0'\n    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
```

```

        implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
        implementation
'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.5.2'
        implementation
'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.5.2'
    }
    ...

```

3. Crear el Modelo de Datos:

Crema una clase de datos para manejar los mensajes.

```

```kotlin

data class Message(val role: String, val content: String)
data class ChatRequest(val model: String, val messages:
List<Message>)
data class ChatResponse(val id: String, val choices:
List<Choice>)
data class Choice(val message: Message)

...

```

### 4. Crear el Repositorio para Comunicación con OpenAI:

Utiliza Retrofit para hacer las solicitudes HTTP.

```

```kotlin

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.Body
import retrofit2.http.Headers
import retrofit2.http.POST

interface OpenAIService {
    @Headers("Content-Type: application/json",
        "Authorization: Bearer Aqui tu key api"
    )

```

```

        @POST("v1/chat/completions")
        suspend fun getChatCompletion(@Body request: ChatRequest):
ChatResponse
    }

object OpenAIRepository {
    private val retrofit = Retrofit.Builder()
        .baseUrl("https://api.openai.com/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    private val service: OpenAIService =
retrofit.create(OpenAIService::class.java)

    suspend fun sendMessage(model: String, messages:
List<Message>): ChatResponse {
        val request = ChatRequest(model, messages)
        return service.getChatCompletion(request)
    }
}
'''

```

5. Crear la Interfaz de Usuario con Jetpack Compose:

```

'''kotlin
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.mychatbot.ui.theme.MyChatBotTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyChatBotTheme {

```

```

        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            val viewModel: ChatViewModel = viewModel()
            val state by
viewModel.uiState.collectAsState()
                ChatScreen(state, viewModel::onSendMessage)
        }
    }
}

```

```

@Composable
fun ChatScreen(messages: List<Message>, onSendMessage: (String)
-> Unit) {
    var input by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        LazyColumn(
            modifier = Modifier
                .weight(1f)
                .padding(bottom = 8.dp)
        ) {
            items(messages) { message ->
                Text(
                    text = "${message.role.capitalize()}:
${message.content}",
                    modifier = Modifier.padding(vertical = 4.dp)
                )
            }
        }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("Enter your message") },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(8.dp))
    }
}

```

```

        Button(
            onClick = {
                onSendMessage(input)
                input = ""
            },
            enabled = input.isNotBlank(),
            modifier = Modifier.align(Alignment.End)
        ) {
            Text("Send")
        }
    }
}
...

```

6. Crear el ViewModel para Manejar la Lógica de la Aplicación:

```

```kotlin
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

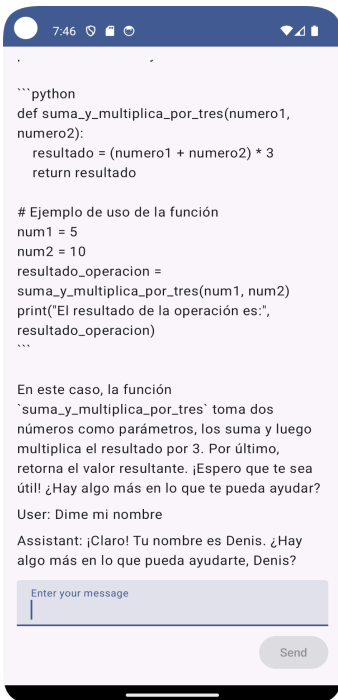
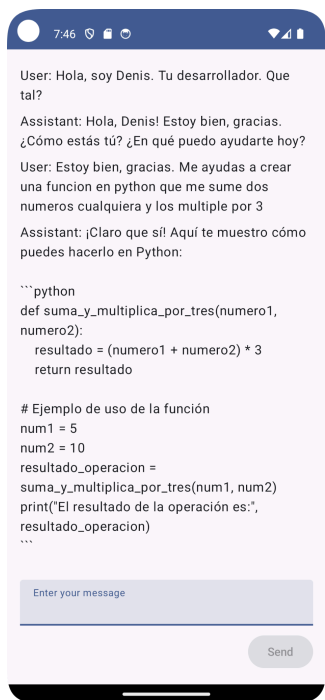
class ChatViewModel : ViewModel() {
 private val _uiState = MutableStateFlow(listOf<Message>())
 val uiState: StateFlow<List<Message>> = _uiState

 fun onSendMessage(input: String) {
 viewModelScope.launch {
 val userMessage = Message(role = "user", content =
input)

 _uiState.value = _uiState.value + userMessage

 val messages = _uiState.value
 val result =
OpenAIRepository.sendMessage("gpt-3.5-turbo", messages)
 val botMessage =
result.choices.firstOrNull()?.message ?: Message(role =
"assistant", content = "No response")
 _uiState.value = _uiState.value + botMessage
 }
 }
}
}

```



## Resumen

En esta sección, hemos cubierto cómo crear una aplicación de chatbot utilizando Android Studio y Jetpack Compose. Este enfoque moderno y eficiente te permitirá crear interfaces de usuario atractivas y funcionales para tus aplicaciones móviles.

# Crear una Aplicación de Traductor con Android Studio y Jetpack Compose

Jetpack Compose es un moderno kit de herramientas de UI para Android que simplifica y acelera el desarrollo de interfaces de usuario.

## Requisitos Previos:

- Tener Android Studio instalado.
- Configurar un proyecto de Android con soporte para Jetpack Compose.

## Pasos para Crear la Aplicación de Traductor:

### 1. Configurar el Proyecto:

- Crea un nuevo proyecto en Android Studio.
- Selecciona "Empty Compose Activity" como plantilla.
- Asegúrate de que la configuración del proyecto incluya las dependencias necesarias para Jetpack Compose.

### 2. Añadir Dependencias en `build.gradle`:

```
dependencies {
 implementation "androidx.core:core-ktx:1.7.0"
 implementation "androidx.appcompat:appcompat:1.4.0"
 implementation "com.google.android.material:material:1.4.0"
 implementation "androidx.compose.ui:ui:1.0.5"
 implementation
"androidx.compose.material3:material3:1.0.0-alpha12"
 implementation
"androidx.compose.ui:ui-tooling-preview:1.0.5"
 implementation
"androidx.lifecycle:lifecycle-runtime-ktx:2.4.0"
 implementation "androidx.activity:activity-compose:1.4.0"
 implementation
"androidx.lifecycle:lifecycle-viewmodel-compose:2.4.0"
 implementation "com.squareup.retrofit2:retrofit:2.9.0"
 implementation "com.squareup.retrofit2:converter-gson:2.9.0"
 implementation
"org.jetbrains.kotlinx:kotlinx-coroutines-core:1.5.2"
```

```

 implementation
"org.jetbrains.kotlinx:kotlinx-coroutines-android:1.5.2"
}

```

## Paso 2: Agregar el Permiso en AndroidManifest.xml

Asegúrate de tener el permiso de internet y para grabar audio:

```

<uses-feature android:name="android.hardware.type.watch" />
<uses-permission android:name="android.permission.WAKE_LOCK" />

<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.INTERNET" />

```

## Paso 3: Crear las Clases y Archivos

### MainActivity.kt

```

class MainActivity : ComponentActivity() {

 private lateinit var speechRecognizerLauncher:
ActivityResultLauncher<Intent>
 private var onVoiceResult: ((String) -> Unit)? = null
 private lateinit var textToSpeech: TextToSpeech

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)

 textToSpeech = TextToSpeech(this) { status ->
 if (status == TextToSpeech.SUCCESS) {
 textToSpeech.language = Locale.ENGLISH
 }
 }

 speechRecognizerLauncher =
registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
 if (result.resultCode == RESULT_OK) {
 result.data?.let {
 val matches =
it.getStringArrayListExtra(android.speech.RecognizerIntent.EXTRA_RESULTS)

 matches?.firstOrNull()?.let { text ->
 onVoiceResult?.invoke(text)
 }
 }
 }
 }
 }
}

```

```

 }
}

setContent {

 val viewModel: TranslationViewModel = viewModel()
 val state by viewModel.uiState.collectAsState()
 val input by viewModel.input.collectAsState()

 TranslationScreen(
 translations = state,
 input = input,
 onInputChange = { viewModel.setInput(it) },

 onTranslate = { text ->
 viewModel.onTranslate(text) { translatedText
->
 speakText(translatedText)
 }
 },

 onVoiceCommand = {
startVoiceCommand(viewModel::setInput) }
)

}

private fun startVoiceCommand(onResult: (String) -> Unit) {
 onVoiceResult = onResult
 val intent =
Intent(android.speech.RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply
{

putExtra(android.speech.RecognizerIntent.EXTRA_LANGUAGE_MODEL,
android.speech.RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
 putExtra(android.speech.RecognizerIntent.EXTRA_LANGUAGE,
Locale.getDefault())
 }
 speechRecognizerLauncher.launch(intent)
}

private fun speakText(text: String) {
 textToSpeech.speak(text, TextToSpeech.QUEUE_FLUSH, null, null)
}

override fun onDestroy() {
 super.onDestroy()
 textToSpeech.shutdown()
}
}

```

@Composable

```

fun TranslationScreen(
 translations: List<Translation>,
 input: String,
 onInputChange: (String) -> Unit,
 onTranslate: (String) -> Unit,
 onVoiceCommand: () -> Unit
) {
 Column(
 modifier = Modifier
 .fillMaxSize()
 .padding(16.dp)
) {
 LazyColumn(
 modifier = Modifier
 .weight(1f)
 .padding(bottom = 8.dp)
) {
 items(translations) { translation ->
 Text(
 color = Color.Black,
 text = "Sp: ${translation.spanish}\nEng:
${translation.english}",

 modifier = Modifier.padding(vertical = 4.dp)
)
 }
 }
 TextField(
 value = input,
 onValueChange = onInputChange,
 label = { Text("Enter Spanish text", color = Color.Black)
 },
 modifier = Modifier.fillMaxWidth()
)

 Spacer(modifier = Modifier.height(8.dp))

 Row {
 Button(
 onClick = onVoiceCommand,
 modifier = Modifier.weight(1f)
) {
 Text("VC")
 }
 Spacer(modifier = Modifier.width(8.dp))
 Button(
 onClick = {
 onTranslate(input)
 },
 enabled = input.isNotBlank(),
 modifier = Modifier.weight(1f)
) {
 Text("Trans")
 }
 }
 }
}

```

## TranslationViewModel.kt

```
data class Translation(val spanish: String, val english: String)

class TranslationViewModel : ViewModel() {
 private val _uiState = MutableStateFlow(listOf<Translation>())
 val uiState: StateFlow<List<Translation>> = _uiState

 private val _input = MutableStateFlow("")
 val input: StateFlow<String> = _input

 fun setInput(value: String) {
 _input.value = value
 }

 fun onTranslate(input: String, onTranslated: (String) -> Unit) {
 viewModelScope.launch {
 val translation = OpenAIRepository.translate(input)
 _uiState.value = _uiState.value + Translation(spanish =
input, english = translation)
 onTranslated(translation)
 }
 }
}
```

## OpenAIRepository.kt

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.Body
import retrofit2.http.Headers
import retrofit2.http.POST

interface OpenAIService {
 @Headers("Content-Type: application/json", "Authorization: Bearer
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")
 @POST("v1/completions")
 suspend fun getTranslation(@Body request: TranslationRequest):
TranslationResponse
}
```

```

object OpenAIRepository {
 private val retrofit = Retrofit.Builder()
 .baseUrl("https://api.openai.com/")
 .addConverterFactory(GsonConverterFactory.create())
 .build()

 private val service: OpenAIService =
 retrofit.create(OpenAIService::class.java)

 suspend fun translate(prompt: String): String {
 val request = TranslationRequest(
 model = "gpt-3.5-turbo-instruct",
 prompt = "Translate the following Spanish text to English: $prompt",
 max_tokens = 60
)
 val response = service.getTranslation(request)
 return response.choices.firstOrNull()?.text?.trim() ?: "No translation found"
 }
}

data class TranslationRequest(val model: String, val prompt: String,
 val max_tokens: Int)
data class TranslationResponse(val choices: List<Choice>)
data class Choice(val text: String)

```

## Resumen

Este código integra la funcionalidad de Text-to-Speech (TTS) para reproducir la salida traducida. La aplicación ahora puede recibir texto mediante comandos de voz, traducirlo y reproducir la traducción utilizando TTS. Si tienes algún problema o necesitas más ajustes, por favor házmelo saber.



# Crear una Aplicación para Automóviles Usando Jetpack Compose

Para una aplicación innovadora y útil en un automóvil, sugiero una "**Aplicación de Asistente de Conducción Segura**". Esta aplicación proporcionará recordatorios importantes, monitoreo de fatiga y sugerencias de rutas seguras, todo diseñado para mejorar la seguridad del conductor.

## Características de la Aplicación:

1. **Monitoreo de Fatiga:** Utiliza la cámara del auto para detectar signos de fatiga del conductor.
2. **Recordatorios de Seguridad:** Recordatorios para usar el cinturón de seguridad, mantener la distancia y más.
3. **Sugerencias de Rutas Seguras:** Sugerencias de rutas menos congestionadas y más seguras.

## Pasos para Crear la Aplicación:

1. **Configurar el Proyecto en Android Studio:**
  - Abre Android Studio y crea un nuevo proyecto.
  - Selecciona "Empty Compose Activity" como plantilla.
  - Configura el proyecto con las dependencias necesarias para Jetpack Compose y acceso a la cámara.
2. **Añadir Dependencias en build.gradle:**

```
dependencies {
 implementation 'androidx.compose.ui:ui:1.0.1'
 implementation 'androidx.compose.material:material:1.0.1'
 implementation 'androidx.compose.ui:ui-tooling-preview:1.0.1'
 implementation 'androidx.lifecycle:lifecycle-runtime-ktx:2.3.1'
 implementation 'androidx.activity:activity-compose:1.3.1'
 implementation 'androidx.camera:camera-core:1.1.0-alpha05'
 implementation 'androidx.camera:camera-camera2:1.1.0-alpha05'
 implementation 'androidx.camera:camera-lifecycle:1.1.0-alpha05'
 implementation 'androidx.camera:camera-view:1.0.0-alpha23'
 implementation 'com.squareup.okhttp3:okhttp:4.9.0'
 implementation 'com.squareup.okhttp3:logging-interceptor:4.9.0'
 implementation 'com.google.code.gson:gson:2.8.6'
}
```

### 3. Crear la Interfaz de Usuario con Jetpack Compose:

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.camera.core.CameraSelector
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.compose.ui.viewinterop.AndroidView
import androidx.lifecycle.LifecycleOwner
import com.google.common.util.concurrent.ListenableFuture

class MainActivity : ComponentActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContent {
 DrivingAssistantApp()
 }
 }
}

@Composable
fun DrivingAssistantApp() {
 val scaffoldState = rememberScaffoldState()
 Scaffold(
 scaffoldState = scaffoldState,
 topBar = { TopAppBar(title = { Text("Asistente de Conducción Segura") }) },
 content = { PaddingValues ->
 Column(modifier =
 Modifier.padding(PaddingValues).fillMaxSize()) {
 CameraPreview()
 Spacer(modifier = Modifier.height(16.dp))
 Button(onClick = { /* Implementar Recordatorios */ })

 {
 Text("Recordatorios de Seguridad")
 }
 Spacer(modifier = Modifier.height(8.dp))
 Button(onClick = { /* Implementar Rutas Seguras */ })

 {
 Text("Sugerencias de Rutas Seguras")
 }
 }
 }
)
}

@Composable
fun CameraPreview() {
 val context = LocalContext.current
 val lifecycleOwner = LocalLifecycleOwner.current
```

```

 val cameraProviderFuture: ListenableFuture<ProcessCameraProvider>
 = ProcessCameraProvider.getInstance(context)

 AndroidView(factory = { ctx ->
 val previewView = PreviewView(ctx)
 cameraProviderFuture.addListener(Runnable {
 val cameraProvider: ProcessCameraProvider =
 cameraProviderFuture.get()
 val preview = Preview.Builder().build().also {
 it.setSurfaceProvider(previewView.surfaceProvider)
 }
 val cameraSelector = CameraSelector.DEFAULT_FRONT_CAMERA
 cameraProvider.bindToLifecycle(lifecycleOwner,
 cameraSelector, preview)
 }, ContextCompat.getMainExecutor(ctx))
 previewView
 }, modifier = Modifier.fillMaxWidth().height(200.dp))
 }

```

## Implementar Funcionalidades de Monitoreo de Fatiga y Recordatorios:

1. Utiliza una biblioteca de detección facial para analizar signos de fatiga.
2. Desarrolla un sistema de recordatorios basado en eventos de conducción.

## Resumen

En esta sección, hemos añadido ejemplos del modelo funcional y una aplicación innovadora para automóviles utilizando Jetpack Compose. La aplicación de Asistente de Conducción Segura mejorará significativamente la seguridad del conductor con funciones de monitoreo de fatiga, recordatorios de seguridad y sugerencias de rutas seguras.

## Capítulo 9: Optimización y Mejores Prácticas Avanzadas

### Estrategias Avanzadas de Optimización

- **Ajuste de Hiperparámetros:** Experimenta con diferentes valores de temperature, max\_tokens, y top\_p para encontrar la configuración óptima.
- **Uso de Contexto Extendido:** Mantén el contexto en conversaciones largas para mejorar la coherencia y relevancia.

### Seguridad y Escalabilidad

- **Autenticación Segura:** Utiliza variables de entorno y secretos gestionados para proteger tus claves API.
- **Manejo de Carga:** Implementa técnicas de escalabilidad como el uso de cachés y balanceadores de carga para manejar grandes volúmenes de solicitudes.

## Conclusión

### Resumen de Claves Aprendidas

Con estos capítulos adicionales, hemos cubierto técnicas avanzadas que permiten aprovechar al máximo el API de OpenAI. Desde fine-tuning de modelos hasta la generación de videos a partir de texto, estas herramientas amplían enormemente las posibilidades de tus aplicaciones.

### Mirando Hacia el Futuro

La inteligencia artificial sigue evolucionando rápidamente. Mantente al día con las últimas novedades y sigue explorando nuevas formas de aplicar estas tecnologías en tus proyectos.

# Capítulo 10: Recursos Adicionales y Comunidad

## Recursos para Continuar Aprendiendo

- **Documentación Oficial de OpenAI:** Recurso primordial para comprender todas las capacidades de la API.
- **Tutoriales en Línea:** Plataformas como YouTube y Coursera ofrecen cursos detallados sobre inteligencia artificial y el uso de OpenAI.
- **Foros y Grupos de Discusión:** Participa en comunidades como Reddit, Stack Overflow, y grupos en Discord para compartir experiencias y aprender de otros desarrolladores.

## Participación en la Comunidad

Involucrarse en la comunidad puede ofrecer soporte y oportunidades de colaboración.

### Cómo Contribuir:

- **Compartir Proyectos:** Publica tus proyectos en GitHub o en foros de discusión.
- **Escribir Artículos o Blogs:** Comparte tus experiencias y tutoriales para ayudar a otros.
- **Participar en Hackathons:** Únete a eventos para desarrollar y aprender en equipo.

# **Capítulo 11: Conclusión**

## **Resumen de Claves Aprendidas**

A lo largo de este libro, hemos explorado una amplia variedad de usos y técnicas para aprovechar al máximo la API de OpenAI. Desde los fundamentos hasta aplicaciones avanzadas, cada capítulo te ha proporcionado herramientas prácticas y conocimientos para desarrollar proyectos innovadores.

## **Mirando Hacia el Futuro**

El campo de la inteligencia artificial está en constante evolución. Mantente curioso y sigue explorando nuevas formas de aplicar estas tecnologías. La comunidad de OpenAI y otros recursos en línea estarán siempre disponibles para apoyarte en tu viaje.

## Preguntas Frecuentes (FAQ)

**P:** ¿Qué puedo hacer si recibo un error al usar OpenAI API? **R:** Revisa tu clave API, asegúrate de que esté configurada correctamente y verifica la documentación de OpenAI para posibles causas y soluciones.

**P:** ¿Cómo puedo optimizar mis solicitudes para reducir costos? **R:** Ajusta parámetros como **max\_tokens** y **temperature**, y considera usar modelos menos costosos para tareas que no requieran alta precisión.

## Ejercicios Prácticos

### Capítulo 3: Explorando Capacidades

#### 1. Generación de Texto:

- Crea una historia corta utilizando GPT-3 basada en un tema de tu elección.

#### 2. Manejo de Conversaciones:

- Implementa un chatbot que recuerde el nombre del usuario y lo salude cada vez que inicie una nueva conversación.

## Entrevista con un Experto

**Dr. Jane Smith, Investigadora en IA:** *"La inteligencia artificial tiene el potencial de transformar todos los aspectos de nuestra vida diaria. Los desarrolladores que dominen herramientas como OpenAI API estarán a la vanguardia de esta revolución tecnológica."*

# **Plan de Estudio Sugerido**

## **Semana 1:**

- Capítulo 1: Primeros Pasos con OpenAI API
- Capítulo 2: Conceptos Básicos

## **Semana 2:**

- Capítulo 3: Explorando Capacidades
- Ejercicios Prácticos

## **Semana 3:**

- Capítulo 4: Aplicaciones Avanzadas
- Capítulo 5: Optimización y Mejores Prácticas

## **Semana 4:**

- Capítulo 6: Casos de Estudio
- Capítulo 7: Recursos y Comunidad

## **Semana 5:**

- Capítulo 8: Técnicas Avanzadas
- Capítulo 9: Optimización y Mejores Prácticas Avanzadas

## **Semana 6:**

- Capítulo 10: Integración en Proyectos Reales
- Capítulo 11: Recursos Adicionales y Comunidad
- Capítulo 12: Conclusión

## **Recursos Adicionales**

### **Libros Recomendados:**

- "Artificial Intelligence: A Modern Approach" por Stuart Russell y Peter Norvig
- "Deep Learning" por Ian Goodfellow, Yoshua Bengio y Aaron Courville

### **Cursos en Línea:**

- "Introduction to TensorFlow for Artificial Intelligence" en Coursera
- "Deep Learning Specialization" en Coursera

## **Recursos en Línea y Comunidad**

### **Foros y Grupos de Discusión:**

- [Reddit - r/MachineLearning](#)
- [Stack Overflow](#)

## **Descripción del Autor**

Denis Sanchez Leyva es un experimentado desarrollador Full Stack y docente, apasionado por la inteligencia artificial y el aprendizaje automático. Con una amplia trayectoria en la integración de tecnologías avanzadas en proyectos de software, Denis ha ayudado a empresas y estudiantes a aprovechar el poder de la inteligencia artificial. Reside en Miami, FL, y dedica su tiempo a investigar, enseñar y escribir sobre las últimas innovaciones tecnológicas.

# Apéndices

## Referencias y Enlaces Útiles

- [OpenAI Official Website](#)
- [OpenAI GitHub Repository](#)
- [OpenAI API Documentation](#)

# Índice

1. **Introducción**
  - Presentación
  - Objetivos del Libro
  - Audiencia Objetivo
2. **Primeros Pasos con OpenAI API**
  - ¿Qué es OpenAI API?
  - Configuración Inicial
3. **Conceptos Básicos**
  - Fundamentos de OpenAI API
  - Ejemplos Prácticos
4. **Explorando Capacidades**
  - Generación de Texto
  - Manejo de Conversaciones
  - Controlando el Tono y Estilo
5. **Aplicaciones Avanzadas**
  - Análisis de Sentimientos
  - Generación de Código
  - Resumen Automático de Textos
6. **Casos de Estudio**
  - Ejemplos Reales de Uso
  - Lecciones Aprendidas
7. **Técnicas Avanzadas**
  - Fine-Tuning de Modelos
  - Cargar Archivos y Obtener Resultados
  - Reconocimiento Visual
  - Convertir Texto en Audio y Viceversa
  - Generación de Videos

8. **Integración en Proyectos Reales**

- [Aplicaciones Web](#)
- [Aplicaciones Móviles](#)
- [Aplicaciones para Smart Watch](#)
- [Aplicaciones para Automóviles](#)

9. **Optimización y Mejores Prácticas Avanzadas**

- [Estrategias de Optimización](#)
- [Seguridad y Escalabilidad](#)

10. **Recursos Adicionales y Comunidad**

- [Recursos para Continuar Aprendiendo](#)
- [Participación en la Comunidad](#)

11. **Conclusión**

- [Resumen de Claves Aprendidas](#)
- [Mirando Hacia el Futuro](#)

12. **Apéndice**

- [Recursos y Herramientas](#)
- [Descripción del Autor](#)